Poznan University of Technology
Faculty of Computing
Institute of Computing Science

Master's thesis

INTERPRETABLE METHODS FOR DATA-TO-TEXT GENERATION

Dominik Maćkowiak, 151915

Supervisor
Mateusz Lango, PhD

Poznań, 2026

Abstract

Contents

Chapter 1

Introduction

1.1 Motivation

Data-to-text (D2T) generation concerns the automatic conversion of structured information into natural-language text. It is used in settings in which the available information is naturally represented as records, tables, databases, or knowledge graphs, but the intended users are more comfortable reading a linguistic description than inspecting the underlying structure. D2T systems must therefore solve two related problems: they must identify and organize the information that should be communicated, and they must express that information in text that is fluent, coherent, and faithful to its source [10, 22].

Recent neural and large language model based systems have substantially improved the fluency and flexibility of generated text. However, these properties do not by themselves guarantee that a system is suitable for applications in which its decisions must be inspected or reproduced. A model may omit information, introduce unsupported statements, or realize the same input differently on separate executions. When the generation process is distributed across model parameters and prompt instructions, it can also be difficult to determine which decision caused an error. These concerns are especially important for structured data, where the generated text should be traceable to explicit source facts and should not silently alter their meaning. Source-aware and semantic evaluation studies have consequently emphasized the need to assess factuality and faithfulness in addition to surface-form similarity [34, 8, 7].

Fully rule-based systems provide a useful contrast. Their behavior can be represented by explicit operations and inspected by a developer, but the rules usually have to be designed manually. This becomes costly when the input contains nested attributes, temporal data, heterogeneous domains, or many possible linguistic realizations. The resulting tension motivates methods that retain an explicit representation of the generation process while reducing the amount of manual engineering required to construct it.

One way to address this tension is to separate meaning representation from surface realization. In this thesis, complex structured instances are first converted into semantic triples, and the triples are then verbalized as natural-language text. The resulting decomposition can be summarized as

$$d \longrightarrow T \longrightarrow y, \tag{1.1}$$

where d denotes a structured data instance, T denotes its semantic representation, and y denotes the generated text. Triples make the facts being communicated explicit and provide an intermediate object that can be inspected, normalized, and evaluated independently of the final wording. Nevertheless, constructing consistent triples from complex data and producing fluent text from those triples remain non-trivial tasks.

The thesis investigates whether large language models can be used to automate the construction of both parts of this process without making the final system entirely opaque. For meaning representation, several LLM-based conversion pipelines and a domain-specialized fine-tuned model are examined. For surface realization, the LLM is not used only to generate the final text for each input. Instead, it proposes modifications to an executable generator that is evaluated and retained through an evolutionary search. This direction is related to the interpretable neurosymbolic NLG approach of Lango and Dušek [17], which uses LLM agents to construct rule-based generators, and to the AlphaEvolve framework, which uses LLMs to propose program changes that are grounded in execution and automatic evaluation [20]. The experiments use OpenEvolve, an open-source implementation of this evolutionary coding approach [27].

The main aim is therefore to develop and evaluate an interpretable D2T pipeline in which the intermediate meaning representation and the surface realizer are explicit artifacts. The work makes the following contributions:

- an investigation of LLM-based methods for converting complex, heterogeneous structured data into consistent semantic triples;
- a comparison of multi-stage conversion pipelines with a domain-specialized model;
- an evolutionary method for constructing an executable surface realizer from semantic triples using LLM-generated program mutations;
- an evaluation of the resulting components using reference-based, semantic, and LLM-based measures, with attention to both generation quality and interpretability.

1.2 Thesis structure

The structure of the thesis is as follows. Chapter 2 introduces the D2T task, reviews the principal generation approaches, discusses interpretability, and describes the evaluation methods used in this area. Chapter 3 presents the meaning-representation stage of the proposed pipeline. It describes how complex structured instances are converted into semantic triples using LLM-based pipelines and a fine-tuned model.

Chapter 4 presents the surface-realization stage. It introduces the AlphaEvolve and OpenEvolve frameworks and explains how an executable program for verbalizing triples is selected, mutated, evaluated, and retained during evolution. Chapter 5 describes the experimental datasets and configurations, reports the results for meaning representation and surface realization, and evaluates the complete pipeline. Chapter 6 discusses the results, limitations, and broader implications of the proposed approach. Finally, Chapter 7 summarizes the thesis and presents its conclusions. The appendices contain the prompts used by the data-conversion methods.

Chapter 2

Data-to-Text Generation

Data-to-text (D2T) generation is a branch of natural language generation in which structured, non-linguistic information is converted into natural-language text. The input can be represented as a table, a database record, a collection of RDF triples, a knowledge graph, or another formal meaning representation. The output is intended for a human reader and should present the relevant facts in a fluent, coherent, and faithful form. D2T systems are therefore situated at the interface between structured data processing and language generation.

2.1 Problem definition

2.1.1 The D2T task

The general aim of D2T generation is to produce a textual description y from structured input x:

$$G(x) = y, \tag{2.1}$$

where G is a data-to-text generator.

D2T is distinguished from text-to-text generation by the nature of its source. In machine translation, summarization, and paraphrasing, the input is already natural language. In D2T, the input is structured and may not have an obvious linguistic order. The generator must therefore construct a discourse from records, attributes, or graph relations before it can realize individual sentences. Common input formats include relational tables, key–value records, RDF graphs, meaning representations, and time series. The required output can range from a short dialogue utterance to a multi-sentence report or a complete document [10, 22].

The generated text is normally expected to satisfy several requirements. It should be *adequate* or *faithful*, meaning that it should express the intended facts without introducing unsupported information. It should provide appropriate *coverage*, so that important input facts are not omitted. It should also be grammatical, fluent, coherent, and appropriately organized. In addition, a practical system should be able to adapt its style, output length, and level of detail to the intended audience. These requirements can conflict: for example, adding more facts can improve coverage while making the text less readable, and maximizing similarity to a reference can encourage wording that does not accurately describe the input.

2.1.2 RDF and semantic triples

The Resource Description Framework (RDF) is a graph-based data model for representing statements about resources [5]. The basic unit of an RDF statement is a triple consisting of a subject, a

predicate, and an object. The subject identifies the entity being described, the predicate denotes a relation or property, and the object identifies another entity or contains a literal value. A compact representation of a statement about a mobile phone can therefore be written as:

$$(\texttt{ex:PhoneModel},\ \texttt{ex:hasBatteryCapacity},\ \texttt{"5000 mAh"}). \tag{2.2}$$

Here, `ex:` is a namespace prefix, `ex:PhoneModel` is the subject, `ex:hasBatteryCapacity` is the predicate, and the string `"5000 mAh"` is the object literal. In a complete RDF representation, resources and predicates are normally identified using globally resolvable identifiers, while literals may carry datatypes or language tags.

A collection of triples forms a directed, labeled graph: subjects and resource objects are nodes, and predicates label the edges between them. Unlike a JSON tree or a database table, this graph does not prescribe an order in which facts must be presented. The same entity can participate in many relations, and an object can simultaneously be the subject of other statements. This makes RDF a useful meaning representation for D2T generation: the triples preserve the facts and their relations while leaving sentence ordering, aggregation, lexicalization, and surface realization to the generation system.

If the input is a set of semantic triples, it can be written as

$$T = \{(s_i, p_i, o_i)\}_{i=1}^{n}, \tag{2.3}$$

where s_i is a subject, p_i is a predicate, and o_i is an object. The generator must produce a text that verbalizes the information encoded in T. The task is not a simple serialization of the triples. Several valid texts can describe the same input, and the system must decide how the facts should be organized and expressed in the target language [10, 22].

The term *semantic triple* is used for a normalized subject–predicate–object statement, usually stored as three strings. Such a triple has the same structural role as an RDF triple and is not a complete RDF document with formal namespaces and datatypes.

2.1.3 Generation stages

Classical descriptions of natural language generation divide D2T generation into a sequence of decisions. Reiter and Dale [25]

1. **Content selection** determines which facts from the input should be included in the output. This decision is necessary when the input contains more information than can be presented in a useful text.

2. **Content ordering** determines the order in which selected facts or entities are introduced. The ordering can follow temporal, spatial, causal, or discourse-based relations.

3. **Content aggregation** groups related facts into clauses, sentences, or paragraphs. Aggregation can reduce repetition and make the relations between facts explicit.

4. **Lexicalization** selects words and phrases that express the predicates and values in the target language.

5. **Referring expression generation** determines how entities should be mentioned, for example by using a proper name on first mention and a pronoun or a shorter description later in the discourse.

6. **Surface realization** converts the resulting plan into grammatical natural-language sentences.

The first two groups of decisions are often summarized by the question "what to say?", whereas lexicalization, referring expression generation, and surface realization address "how to say it?". The distinction is useful even when all stages are implemented by a single neural model. It identifies the different sources of errors and makes it possible to introduce intermediate representations that can be inspected or constrained [19].

2.1.4 Properties and challenges

The difficulty of D2T generation depends on the input representation, the number of facts, the required output length, and whether the system is allowed to select only part of the input. Short inputs with a small fixed set of attributes can often be handled by templates or simple sequence-to-sequence models. Larger inputs require content planning, aggregation, and mechanisms for maintaining consistency over several sentences.

Faithfulness is a particularly important challenge. A system can generate a grammatical sentence that contains an incorrect value, omits an input fact, or adds information that is not supported by the source. Such errors are often called hallucinations. Neural systems may be fluent while still making these errors, especially when the input is long or contains many similar records [34, 8].

Another challenge is the multiplicity of valid realizations. A single set of triples can be expressed with different word choices, sentence boundaries, and fact orderings. A metric that rewards only agreement with one reference can therefore penalize a valid output. This issue affects both training and evaluation, and motivates the use of multiple references, source-aware metrics, and human assessment [7, 21].

2.2 Methods

2.2.1 Rule-based and template-based systems

The earliest D2T systems were built as manually engineered pipelines. A domain expert defined rules for selecting and ordering facts, templates for common sentence structures, and lexicalization procedures for inserting values into those templates. A template such as "⟨entity⟩ is located in ⟨place⟩" can be instantiated whenever the input contains a corresponding location relation.

The modular structure of these systems offers several advantages. The rules can be inspected directly, unsupported facts can be excluded by construction, and the output style can be controlled precisely. Runtime generation is also fast, because it does not require a large neural model. These properties made templates attractive in domains such as weather forecasting, sports reporting, and dialogue systems [25, 10].

Their main limitation is the cost of domain engineering. New predicates, entities, combinations of facts, and linguistic constructions require new rules or templates. A finite template inventory also restricts lexical and syntactic variation. As the domain becomes more varied, maintaining interactions between content selection, aggregation, referring expressions, and surface realization becomes difficult. The resulting systems can be faithful and controllable while still sounding repetitive or unnatural.

2.2.2 Retrieval and statistical systems

Retrieval-based systems select an existing sentence or template that resembles the input and adapt it to the current values. They reduce the amount of handwritten linguistic knowledge, but their coverage is limited by the examples available in the retrieval collection. They can also produce awkward results when no sufficiently similar example exists.

Statistical systems learn generation decisions from aligned data rather than encoding every decision manually. Probabilistic models can learn, for example, which facts tend to be mentioned together, which order is likely, and which phrases correspond to particular attributes. Hidden Markov models, statistical machine translation, and learned alignment models have all been used for this purpose [30, 22].

Compared with fully handwritten systems, statistical methods can generalize to combinations that were not explicitly covered by a template. However, their behavior is still dependent on the quality and size of the aligned corpus. The probabilistic decisions are less transparent than explicit rules, and errors in alignment or sparse observations can lead to omissions, incorrect lexicalization, or incoherent ordering.

2.2.3 Neural sequence-to-sequence systems

Neural D2T systems formulate generation as conditional language modeling. The input representation is encoded into a vector sequence and a decoder generates the output one token at a time. Recurrent encoder–decoder models with attention were among the first widely used neural architectures. Attention allows the decoder to focus on different input records while generating different parts of the text.

The input may be linearized into a sequence, or it may be represented using a specialized encoder. For RDF and graph inputs, graph neural networks can encode nodes and labeled edges before decoding a textual sequence. For tables and records, hierarchical encoders can separately represent rows, fields, and cell values. Copy mechanisms allow values such as names, dates, and numbers to be copied from the input instead of generated from the vocabulary. This is useful when exact values are rare or unseen during training.

Longer documents introduce additional problems. The decoder must remember which records have already been mentioned, maintain a coherent discourse order, and avoid repeating or omitting facts. The RotoWire data-to-document study showed that neural models can produce fluent text while struggling with content selection and long-range structure; in some settings, templated baselines performed better on content-related measures [34].

2.2.4 Modular neural and end-to-end architectures

Neural D2T systems can be organized as a modular pipeline or as an end-to-end model. In a modular neural pipeline, separate models perform decisions such as discourse ordering, text structuring, referring expression generation, lexicalization, and realization. The output of one module is passed to the next one. In an end-to-end system, the input is encoded and directly decoded into text, with no explicitly supervised intermediate plan.

Castro Ferreira et al. [3] compared neural pipeline and end-to-end systems for RDF-to-text generation using GRU and Transformer architectures. Their results indicate that explicit intermediate steps can improve generation quality and generalization to unseen inputs. The pipeline also makes individual decisions easier to analyze, although errors can propagate between modules and the system requires intermediate annotations or procedures for constructing them.

An intermediate design is provided by Moryossef et al. [19]. Their system first creates a symbolic text plan that specifies the division of facts into sentences and their ordering, and then uses a neural model to realize that plan. The symbolic planning stage improves control over coverage and structure, while the neural realization stage supplies linguistic flexibility. This division illustrates that interpretability and neural generation do not have to be mutually exclusive.

2.2.5 Transformers, pretrained models, and controllable generation

Transformer architectures replaced recurrent sequence models in many D2T systems because self-attention provides a direct mechanism for relating distant input and output positions. Pretrained language models can then be adapted to D2T through supervised fine-tuning, prompt-based generation, or parameter- efficient adaptation. These models provide strong linguistic fluency and can transfer knowledge between domains, but their internal generation decisions are difficult to inspect.

Several techniques are used to improve faithfulness and controllability. Copy and coverage mechanisms encourage the model to preserve input values and avoid repeating already realized content. Auxiliary reconstruction or attribute prediction objectives provide additional supervision about the source. Content planning separates high-level decisions from token generation, and reranking can select a more faithful output from a set of candidates. Constrained decoding can also prevent invalid values or enforce a required output structure. These techniques address specific failure modes, but they usually add specialized components and do not fully expose the learned realization policy.

2.3 Interpretable D2T Approaches

Interpretability in D2T generation concerns the ability to understand why a particular text was produced from a particular input. An interpretable system should make the connection between source facts, generation decisions, and output phrases sufficiently explicit for a researcher or domain expert to inspect. Related properties include controllability, faithfulness, auditability, and the ability to modify a behavior without retraining the whole system.

2.3.1 Explicit rules and templates

Handwritten rule-based systems are the most direct example of interpretable D2T. The relationship between a predicate and its realization is represented by a visible rule or template. Content selection and ordering can be examined as conditions in the generation procedure, and values copied into the output can be traced to their source fields. If a system produces an unsupported fact, the responsible rule can often be identified directly.

This transparency is obtained at the cost of manual engineering. The number of rules grows with the number of predicates, fact combinations, discourse patterns, and linguistic variants. A rule base may therefore be highly interpretable but difficult to extend to a new domain. It may also fail to provide the lexical and syntactic variety expected from natural text.

2.3.2 Intermediate representations and hybrid systems

Modular pipelines expose intermediate objects such as ordered triples, sentence plans, dependency structures, or lexicalization templates. These objects make the generation process easier to analyze than a single undifferentiated neural decoder. The symbolic planning component of Moryossef et

al. [19] is an example: the selected facts and their sentence structure can be verified before neural realization takes place.

Hybrid systems combine explicit rules with learned components. For example, a rule-based content planner can guarantee that selected facts are represented, while a neural realizer can provide more varied language. Conversely, a neural model can propose candidate structures that are filtered by a symbolic faithfulness checker. These designs improve specific aspects of transparency, but the learned parts remain difficult to explain and the boundaries between modules can introduce additional engineering complexity.

2.3.3 LLM-generated rule-based systems

Recent work has investigated a different division of labor between language models and D2T systems. Lango et al. [33] use a large language model to implement a rule-based D2T system in Python. The language model is used during system construction, while the resulting program performs the actual generation at inference time. This approach retains an inspectable code artifact, operates without a GPU during inference, and offers a way to compare program-based generation with direct prompting and neural fine-tuning.

Lango and Dusek [17] extend this general direction by using interactions between LLM agents to construct an interpretable rule-based RDF-to-text generator. Their results show that an LLM can be used to build an NLG system rather than only to produce each output directly. The generated code can be inspected and executed independently of the model that created it.

These approaches address the opacity of direct neural generation, but they leave an important question open: how should the generated program be improved when its initial rules are incomplete, incorrect, or overly specialized? A single construction procedure or a fixed sequence of agent interactions may not explore the space of alternative program structures systematically. Manual correction can restore quality, but it reduces the automation that motivated program generation in the first place.

2.4 Performance Evaluation

2.4.1 Evaluation dimensions

No single measurement captures all desirable properties of generated text. Evaluation is therefore usually organized along several dimensions. *Text quality* includes grammaticality, fluency, readability, and coherence. *Semantic adequacy* or *faithfulness* concerns whether the output expresses the source data correctly. *Coverage* concerns omissions, while *non-redundancy* concerns repetitions. Some applications additionally evaluate style, diversity, latency, and computational cost.

Evaluation procedures can be reference-based or reference-free. Reference-based metrics compare a generated text with one or more human-written texts. They are easy to apply when a corpus contains references, but they can penalize valid paraphrases and cannot by themselves determine whether a reference or a candidate is supported by the source. Reference-free procedures compare the output directly with the input data or ask human annotators to assess the input–output pair. They are better suited to factuality and coverage, but they require a model or annotation procedure capable of interpreting the structured input.

2.4.2 Reference-based automatic metrics

BLEU

BLEU (Bilingual Evaluation Understudy) was introduced for machine translation by Papineni et al. [24] and became one of the most frequently reported metrics in D2T research. It measures modified precision for matching unigrams, bigrams, trigrams, and four-grams between a candidate and one or more references. A brevity penalty reduces the score of candidates that obtain high precision by being much shorter than the references. A simplified corpus-level form is

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right), \tag{2.4}$$

where p_n is the modified precision for n-grams, w_n are the weights, and BP is the brevity penalty.

BLEU is inexpensive, deterministic, and useful for comparing systems under the same implementation and preprocessing conditions. Its limitations are equally important. It rewards lexical overlap rather than meaning, does not directly measure factual correctness, and can assign a low score to a faithful paraphrase. It can also assign a relatively high score to a text that shares surface forms with a reference while omitting or altering source facts.

METEOR

METEOR was proposed by Banerjee and Lavie [2] to improve the relationship between automatic and human evaluation. It aligns candidate and reference unigrams using exact matches and, depending on the implementation, stem and synonym matches. Precision and recall are combined in a weighted harmonic mean, and a fragmentation penalty discourages matches that occur in a disordered fashion. METEOR can therefore recognize some lexical variation that BLEU cannot. It remains reference-dependent, however, and its behavior depends on the available stemming and synonym resources.

BERTScore

BERTScore uses contextual embeddings rather than exact token identity to compare a candidate with a reference [36]. Each token in one text is matched to semantically similar tokens in the other text using cosine similarity. Precision, recall, and F1 variants can be reported. Because contextual embeddings represent related words and paraphrases more flexibly, BERTScore can capture similarities missed by n-gram metrics. It is nevertheless dependent on the pretrained model, tokenizer, language, and reference text, and semantic similarity does not guarantee factual faithfulness to the source data.

BLEURT

BLEURT is a learned evaluation metric trained to predict human judgments of generated text [26]. It combines pretrained language-model representations with further training on synthetic perturbations and human ratings. Unlike a direct n-gram count, BLEURT can learn that some substitutions, omissions, or fluency problems affect quality differently. This flexibility can improve correlation with human judgments, but it also introduces dependence on the metric model and its training distribution. Domain shift, model bias, and the use of a reference remain relevant limitations.

2.4.3 Source-aware and semantic evaluation

Reference-based metrics treat the reference as the principal target. In D2T, the structured source should also be treated as an evaluation object. This is especially important when a reference contains information that is not present in the input or when a candidate expresses an input fact using a different wording.

PARENT (Precision And Recall of Entailed N-grams from the Table) addresses this problem for table-to-text generation [7]. It compares the candidate with both the reference and the source table. Its precision and recall calculations use entailment estimates to reward information supported by the table and to reduce the effect of divergent reference text. PARENT illustrates the value of task-specific metrics, although its original formulation is designed for semi-structured tables and requires a suitable source-to-text entailment procedure.

Dušek and Kasner [8] proposed an NLI-based semantic-accuracy metric for D2T. The structured input is converted into simple textual statements, after which natural-language inference is used in both directions. Testing whether the input entails the output helps detect unsupported information, while testing whether the output entails each input fact helps detect omissions. The resulting labels can distinguish faithful outputs, omissions, hallucinations, and combinations of these errors. This approach directly targets the semantic requirements of D2T, but its accuracy depends on the data-to-text conversion, the NLI model, and the assumptions about whether all input facts should be realized.

2.4.4 Human evaluation

Human evaluation remains necessary because automatic scores are proxies for quality. Annotators can assess the generated text directly or judge it together with the input representation and reference. Typical criteria include grammaticality, fluency, coherence, informativeness, semantic adequacy, coverage, repetition, and factual consistency. For D2T, semantic criteria are often operationalized through binary questions such as whether all required facts are present and whether unsupported facts have been added. Text quality can also be measured on a Likert scale.

Human evaluation is expensive and can be affected by annotator expertise, instructions, sample selection, and the number of annotators per item. For this reason, human-evaluation studies should report inter-annotator agreement and describe the assumptions behind the selected statistic. The survey by Artstein and Poesio [1] discusses the use of agreement coefficients in computational linguistics. Cohen's kappa measures chance-corrected agreement between two annotators assigning nominal categories, whereas Krippendorff's alpha is designed to accommodate multiple annotators, missing annotations, and different measurement levels [4, 15]. A reliable evaluation should specify the criteria, scale, annotation procedure, number of annotators, agreement measure, and aggregation method. Human assessment is particularly important when automatic metrics disagree or when the task permits many valid verbalizations. [32, 35]

2.4.5 Metrics used

The evolutionary evaluator used in this work computes BLEU against the reference lexicalizations during program evolution. The final evaluation also reports METEOR and BLEURT, allowing lexical, alignment-based, and learned reference-based comparisons. The experiment additionally calculates SENLEN, a project-specific structural diagnostic. It counts sentence boundaries using periods and returns the absolute difference between the candidate sentence count and the average

reference sentence count:

$$\mathrm{SENLEN}(y) = \left| \#\mathrm{sentences}(y) - \frac{1}{|R|} \sum_{r \in R} \#\mathrm{sentences}(r) \right|. \tag{2.5}$$

Here, R is the set of reference lexicalizations associated with the input instance. The implementation approximates the number of sentences by splitting each text at full stops. Unlike BLEU, METEOR, and BLEURT, a lower SENLEN value indicates greater structural proximity and the measure is not a general semantic quality metric. It is therefore treated as a diagnostic rather than as a substitute for faithfulness or human evaluation.

These metrics serve different purposes. BLEU, METEOR, and BLEURT measure similarity to reference texts, whereas SENLEN measures a simple structural property. None of them alone proves that every input triple has been verbalized or that no unsupported fact has been introduced. Their results must therefore be interpreted together with source-aware analysis and qualitative inspection of generated examples. The next chapters describe how these measurements are incorporated into the evolutionary surface-realization system and how the resulting programs are evaluated experimentally.

Chapter 3

Building interpretable meaning representations from complex data

3.1 Method Overview

The central problem addressed in this work is the conversion of complex structured data into interpretable meaning representations. Modern data sources frequently provide information in structured formats such as JSON or XML, containing nested fields, time series, categorical values, and hierarchical relationships. While these formats are efficient for data storage and exchange, they are not directly amenable to semantic reasoning, or natural language understanding tasks. The challenge lies in extracting the semantic content from these structures and representing it in a standardized, machine-interpretable form.

The goal is to transform structured data into semantic triples—subject-predicate-object statements that capture the essential meaning of the data. For example, consider a JSON object describing a mobile phone with nested specifications:

```
{"name": "PhoneModel", "display": {"size": "6.5 inch", "resolution": "2400x1080"},
"battery": {"capacity": "5000 mAh"}, "camera": {"main": "108 MP"}}
```

This structured data should be converted into triples such as:

- `(PhoneModel, hasDisplaySize, 6.5 inch)`
- `(PhoneModel, hasDisplayResolution, 2400x1080)`
- `(PhoneModel, hasBatteryCapacity, 5000 mAh)`
- `(PhoneModel, hasMainCamera, 108 MP)`

These triples capture the semantic relationships in a format that can be directly used for querying, and reasoning.

This transformation presents several challenges that must be addressed by any practical method. First, the input data varies significantly in structure and complexity: some sources contain flat key-value pairs, while others have deeply nested hierarchies, time series with dozens of data points, or categorical attributes with multiple values. Second, the extraction must preserve semantic accuracy, ensuring that the generated triples faithfully represent the information in the source data without introducing errors or omissions. Third, predicate normalization is essential for consistency: the same relationship should be expressed using the same predicate across different instances (e.g., always `hasBatteryCapacity` rather than sometimes `battery_capacity`, sometimes `has_battery`,

etc.). Fourth, the methods must scale to handle large datasets efficiently, as real-world applications may involve thousands or millions of instances.

To address these challenges, two complementary approaches are investigated in this work. The first consists of pipeline-based methods that use large language models in a multi-stage process: first generating natural-language references from the structured data, then extracting semantic triples from those references. The second approach distills this behavior into a single domain-specialized fine-tuned model that produces both the natural-language text and the semantic triples in a single pass. Both approaches are described in detail in the following sections.

3.1.1 Conversion Methods Overview

Two complementary approaches are investigated for converting raw structured instances into semantic triples. The first consists of three pipeline variants that use a general-purpose LLM to first generate reference texts and then extract triples, while the second distills this behavior into a single domain-specialized fine-tuned model.

Pipeline-based methods. The first approach, described in Section 3.2, consists of three pipeline variants that convert data to triples using large language models. All three methods follow a common two-stage structure: in the first stage, each raw structured instance is converted into a natural-language reference text; in the second stage, semantic triples are extracted from that text. The three variants differ in how they manage predicate consistency and validation.

The fundamental challenge is that predicate consistency—ensuring the same semantic relationship is expressed with the same predicate across all instances—requires some form of coordination between extraction calls, but such coordination introduces sequential dependencies that reduce throughput. A fully parallel approach maximizes speed but produces inconsistent predicates; a fully sequential approach ensures consistency but is slow. The three variants explore different points in this tradeoff space:

- **Text-First Extraction with Normalization** prioritizes throughput by extracting triples independently in parallel, accepting predicate inconsistency during extraction and resolving it through a post-hoc normalization stage. This approach is the fastest but requires additional LLM calls for normalization.
- **Blueprint-Iterative Refinement** prioritizes interpretability and control by creating an explicit, human-inspectable specification (the blueprint) that defines the predicate vocabulary upfront. The blueprint is iteratively validated and refined against reference texts before extraction. This approach provides the most structured and auditable output but requires multiple validation rounds.
- **Evolving Catalog** seeks a middle ground by building predicate consistency incrementally during a sequential bootstrap phase, then switching to batch processing once the predicate vocabulary stabilizes. This hybrid approach balances consistency and throughput, adapting to the data's complexity.

Fine-tuned model. The second approach, described in Section 3.3, distills the behavior of the multi-stage pipeline into a single domain-specialized model. Using *QLoRA* (Quantized Low-Rank Adaptation), a model is fine-tuned to produce both natural-language text and semantic triples in a single decode pass. This approach trades the flexibility of the pipeline for reduced inference

latency and improved consistency, at the cost of requiring per-domain fine-tuning on reference outputs produced by the pipeline.

3.2 Data conversion with LLMs

Pipeline overview. All three methods follow a common two-stage structure. In the first stage, each raw structured instance is converted into a natural-language reference text. In the second stage, semantic triples are extracted from that text. This text-first approach decouples verbalization from triple extraction, allowing each stage to be optimized independently and batched for throughput. The rationale for this decomposition is that natural language serves as an intermediate representation that generalizes the input data, while the structured triple output can be directly used for downstream tasks such as program synthesis.

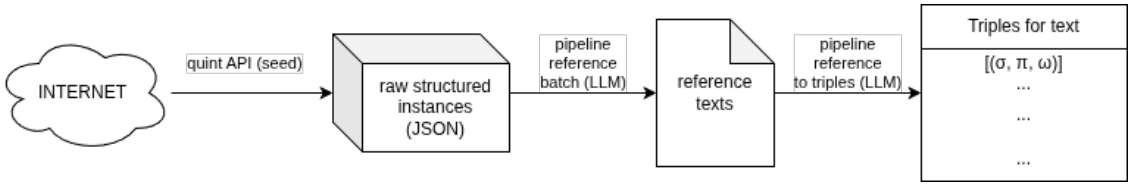

Figure 3.1: General pipeline flow: raw structured data from quintd is first converted to reference texts, then to semantic triples using one of the three pipeline methods.

3.2.1 Text-First Extraction with Normalization

Text-First Extraction with Normalization decomposes the data-to-triples problem into two fully batched LLM stages and a normalization stage. The method deliberately avoids any inter-instance coupling during extraction: each reference and each triple set is produced independently and in parallel, which makes the method stateless, at the cost of not enforcing predicate consistency during extraction.

Stage 1 — Reference text generation. In the first stage the model is invoked once per instance under a `text-generation system prompt`, P_{txt} (see Appendix A.3), whose objective is to convert one structured data instance into concise natural language and to return the result as a JSON object of schema `{"text": "..."}`.[1] The prompt further instructs the model to capture the most important information, trends, extremes, and notable conditions, and to summarize time series (e.g., weather forecasts) at a high level rather than listing every data point.

All N requests are packaged as a single OpenAI Batch API job and a single request outputs t_i, where t_i is the generated reference text for instance d_i. This batched stage yields, for every instance, a natural-language reference t_i that constitutes the intermediate representation from which triples are subsequently derived.

Stage 2 — Triples-from-text extraction. A second OpenAI Batch is then assembled in which every reference t_i is paired with a `triples-from-text system prompt`, P_{tr} (see Appendix A.3), whose objective is to extract semantic triples from natural language text, returning JSON of schema `{"triples":["subject":"...","predicate":"...","object":"..."]}`. The prompt imposes two constraints: it instructs the model to preserve the full meaning of the input text and to keep predicates in `lower_snake_case` when possible, and to avoid duplicate triples.

[1] Throughout this work, JSON schemas are presented in simplified notation with placeholder values (e.g., `"text": "..."`) for readability, rather than as formal JSON Schema definitions.

Each instance is therefore an independent text-to-triples problem, and the batch can be fully parallelized. The output of a single request is T_i, where T_i is a set of triples that describe the reference text t_i.

Stage 3 — Post-hoc predicate normalization. Because Stage 2 imposes no inter-instance vocabulary control, this stage invokes a predicate-normalization routine. The routine operates on the flattened list of triples and iteratively merges semantically equivalent predicates:

Let Π be the set of distinct predicates present in the extracted triples. A Union-Find structure over Π is initialized with every predicate in its own singleton class. A *Union-Find* (disjoint-set) is a data structure that maintains a collection of disjoint sets and supports two operations: `find(x)` returning the representative element of the set containing x, and `union(a, b)` merging the sets containing a and b. A *singleton class* is an equivalence class containing only a single element.

For each round $r = 1, 2, \ldots$ (bounded by $2|\Pi|$):

- The model is prompted with the current set of canonical predicates (class representatives) and asked to propose candidate equivalent pairs under a `candidate-pairs system prompt` (see Appendix A.3).
- If no candidate pairs are returned, or if the candidate set is identical to one proposed in an earlier round (indicating the model has converged or is cycling without progress), the loop terminates.
- Otherwise, every candidate pair is sent to the model under a `strict-equivalence system prompt` (see Appendix A.3) that returns a JSON object with three fields: `equivalent` (boolean), `confidence` (a score between 0 and 1), and `reason` (a textual explanation). The model is instructed to judge equivalence strictly (mergeable in a knowledge graph), not mere relatedness.
- Pairs judged equivalent are merged in the Union-Find structure. If a round produces no merges, the loop terminates.

After termination, each predicate is mapped to the canonical representative of its class, defined as the first member of the class (deterministic, order-based choice). The triples are then rewritten with these canonical predicates.

In the normalization step, the final output combines: (i) generated references, (ii) extracted triples, and (iii) normalized triples alongside the canonical mapping and the comparison audit trail.

3.2.2 Blueprint-Iterative Refinement

Blueprint-Iterative Refinement introduces an explicit notion of extraction blueprints that act as an intermediate, human-inspectable artifact between the domain declaration and the final triple extraction. An *extraction blueprint* is a structured specification containing predicate labels, example triples, example text, and transformation guidelines. The blueprints are generated top-down from the domain name, then iteratively validated and revised against the generated references until it is confirmed that they can extract triples for every instance. Only then is a single batched triple-extraction run executed under the finalized blueprint.

Stage 1 — Initial blueprint generation. Given the user-supplied problem domain string δ (e.g., weather forecast), the model is invoked once under a `blueprint-creation system prompt`, $P_{\text{blueprint}}$ (see Appendix A.4), and asked to design an extraction blueprint for a domain. Its

response is constrained by a strict JSON schema to contain exactly four fields:

$$R = \Big(\Pi_R,\ E,\ T_{\text{ex}},\ G\Big),$$

where Π_R is a list of predicate labels, E is a list of example triples $\{(\sigma_k, \pi_k, \omega_k)\}$ using those predicates, T_{ex} is a short natural-language passage consistent with the example triples, and G is a list of free-text transformation guidelines. The response is sanitized, which deduplicates predicates and guidelines, discards malformed example triples, and falls back to the corresponding field values from the previous blueprint R_{r-1} for any field that is empty or malformed (for the initial blueprint R_0, empty fields default to empty lists or strings). The resulting R_0 constitutes the initial blueprint. Conceptually, R_0 is a self-contained specification of how the model believes instances of the domain should be verbalized and triplelized, complete with concrete formatting conventions embedded in the example triples.

Stage 2 — Reference text generation. The reference-generation batch is identical to that of the previous method: every instance d_i is sent to the `text-generation system prompt` P_{txt} within a single OpenAI Batch, producing references t_i.

Stage 3 — Blueprint validation and iterative revision. The core of the method is an iterative loop, bounded by $R_{\max}$. The loop maintains a current blueprint R and operates as follows for each round $r = 1, \ldots, R_{\max}$:

1. **Feasibility scan.** A random reference text with replacement t_i is selected and the model is invoked under a `blueprint-feasibility system prompt`, P_{feas} (see Appendix A.4), which receives the current blueprint R and the text t_i, and returns a JSON object with three fields: `possible` (boolean indicating whether the blueprint is sufficient), `reason` (textual explanation), and `blueprint_gaps` (a list of descriptions of what is missing from the blueprint in order to extract triples from t_i).

2. **Pass condition.** If the scan reaches the $R_{\max}$ without encountering a `possible = false` judgment, the blueprints are declared validated against all texts and the loop terminates.

3. **Failure and revision.** As soon as a reference t_{i^*} is judged infeasible under R, the scan halts. The `blueprint-revision system prompt`, P_{rev} (see Appendix A.4), is then invoked with the domain δ, the current blueprint R, the failing text t_{i^*}, the failure reason, and the enumerated `blueprint_gaps`. Its response is again constrained to the same four-field schema structure $R = (\Pi_R, E, T_{\text{ex}}, G)$ described in Stage 1. The model is instructed to keep existing high-quality blueprint components, but add or adjust what is necessary to cover the failing case. The sanitized result replaces R as the new current blueprint.

4. **Restart.** Because revising the blueprint may break previously validated instances or expose new gaps, the scan is restarted and the round number r set to 1. The loop thus monotonically accumulates coverage, but backtracks the scan position whenever the blueprint is modified.

If the budget $R_{\max}$ is exhausted before all texts are validated, the method emits a warning and proceeds to final extraction with the best-effort blueprint; the output records that validation was incomplete.

This stage implements a generate-test-revise loop in which the blueprint is the artifact being refined, the reference texts act as test cases, and the feasibility prompt acts as a model-internal test oracle.

Stage 4 — Final batched extraction under finalized blueprint. Once the validation loop has terminated, a single OpenAI Batch is submitted in which every reference t_i is paired

with a `triples-from-text-with-blueprint system prompt`, $P_{\text{tr\&}R}$ (see Appendix A.4). This prompt instructs the model to extract semantic triples from the natural language text using the provided blueprint, to follow the blueprint guidelines, to mimic the formatting style of the example triples, and to prefer predicates from the blueprint predicate list whenever possible. The blueprint R is serialized and injected in each request. The batch output is parsed into T_i.

3.2.3 Evolving Catalog

Evolving Catalog is a hybrid text-first data-to-triples pipeline that combines an evolving predicate catalog with a stability-triggered transition from sequential to batch processing. It is designed to keep the predicate vocabulary consistent across instances while reducing the latency overhead of per-instance, order-dependent LLM calls.

Stage 1 — Reference text generation. An OpenAI Batch is assembled in which every instance d_i is paired with the `text-generation system prompt` P_{txt} (see Appendix A.5), whose purpose is to produce a concise natural-language summary that captures the most important information, trends, extremes, and notable conditions, and summarizes time series at a high level. The batch output is parsed into text reference t_i.

Stage 2 — Sequential catalog-driven triple extraction. The second stage maintains a mutable predicate catalog C, initialized as empty. The catalog is a set of entries $\{(\pi, e_\pi)\}$, where each entry associates a predicate label π with an example triple $e_\pi = (s, \pi, o)$ in which that predicate was first observed. The catalog thus carries both a normalized vocabulary and a small set of usage examples that serve as in-context guidance for the extraction model.

The instances are processed strictly in order:

1. **Bootstrap instance ($i = 0$).** Because C is empty, the reference t_0 is sent to the model under a `first-instance system prompt`, P_{first} (see Appendix A.5), that simply asks for semantic triples from the text with concise `lower_snake_case` predicates and no duplicates. This yields a set of triples T_0.

2. **Catalog-guided instances ($i \geq 1$).** For each subsequent instance d_i, the reference t_i is sent under a `catalog-guided system prompt`, P_{cat} (see Appendix A.5), in which the current catalog C is serialized and injected into the user prompt. The model is instructed to use only predicates from the provided catalog when possible and to introduce a new predicate only when none of the catalog predicates can describe the fact. This yields triples T_i.

   After each instance, every triple in T_i is inspected. If a triple contains a predicate $\pi \notin C$, a new entry (π, e_π) is added to C, where e_π is set to that triple (the first occurrence).

Stage 3 — Stable-window heuristic and batched tail processing. A counter w tracks the number of consecutive instances that did not introduce any new predicate; it is reset to zero whenever a new predicate is added. Let W denote the stable window parameter. The sequential phase terminates as soon as $w \geq W$ and at least one instance remains unprocessed. Let i^* be the index at which this condition is first met; the catalog C at that point is declared frozen.

All instances $d_{i^*+1}, \ldots, d_N$ (the tail) are then processed in a single second OpenAI Batch, again under P_{cat} and with the frozen catalog C injected into every request. Two design choices are important here:

- The tail is parallelized, hence the cost of the remaining $N - i^* - 1$ LLM calls is amortized into one batch.

- Predicates that first appear within the tail are not added back to C. The catalog is finalized at the moment of stability, which guarantees that the predicate vocabulary presented to a human evaluator is fully determined by the sequential phase and does not silently grow during batch processing.

3.3 Fine-tuned model for data conversion

The methods described in Section 3.2 rely on repeated invocations of a general-purpose large language model at inference time: each instance requires one or more LLM calls to produce reference text, extract triples, and, in some pipelines, to validate or revise intermediate artifacts. While this design keeps the pipeline flexible, it incurs substantial latency and computational cost, particularly when the same domain is processed repeatedly. An alternative is to *distill* the behaviour of the multi-stage pipeline into a single, domain-specialized model that produces both the natural-language reference and the corresponding triple set in one decode pass. This section describes how such a model is obtained via supervised fine-tuning with QLoRA.

3.3.1 Supervised Fine-Tuning and QLoRA

Supervised fine-tuning (SFT) adapts a pre-trained language model to a specific task by training it on input–output pairs. Given a pre-trained model M_θ with parameters θ and a dataset $\{(x_i, y_i)\}_{i=1}^{n}$ of input–output examples, SFT minimizes the negative log-likelihood of the targets [23]:

$$\mathcal{L}(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{t=1}^{|y_i|}\log P_\theta(y_{i,t} \mid x_i, y_{i,<t}). \tag{3.1}$$

For small models (up to a few billion parameters), full SFT—updating all parameters θ—is feasible on modern GPU hardware. However, for models with tens of billions of parameters, full fine-tuning becomes prohibitively expensive: a 31-billion-parameter model in 16-bit precision requires approximately 62 GB of memory for the weights alone, and the standard Adam optimizer [14] maintains two additional moment estimates per parameter, roughly doubling the memory footprint.

Parameter-efficient fine-tuning (PEFT) methods address this by keeping the base model weights frozen and introducing a small number of trainable parameters [18]. Among these, *Low-Rank Adaptation* (LoRA) [11] has become the most widely adopted. LoRA decomposes the weight update into a low-rank product: for each frozen weight matrix W_0, the forward pass is modified as

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} BAx, \tag{3.2}$$

In this formulation, $W_0 x$ represents the output of the original pretrained model, while $\Delta W x$ is the learned task-specific adaptation. The key insight from Hu et al. [11] is that the weight updates ΔW during fine-tuning also have low "intrinsic rank," meaning they can be approximated by a low-rank decomposition. Rather than updating the full weight matrix W_0, LoRA keeps W_0 frozen and learns the low-rank matrices A and B, where $\Delta W = \frac{\alpha}{r} BA$. The scaling factor $\frac{\alpha}{r}$ controls the magnitude of the adaptation relative to the pretrained weights. This decomposition reduces the number of trainable parameters from $d \times k$ (the full weight matrix) to $r \times (d + k)$ (the two low-rank matrices), where $r \ll \min(d, k)$.

where:

- $W_0 \in \mathbb{R}^{d \times k}$ is the frozen pretrained weight matrix
- d is the output dimension and k is the input dimension of the layer

- $x \in \mathbb{R}^k$ is the input vector to the layer
- $h \in \mathbb{R}^d$ is the output (hidden state) of the layer
- $A \in \mathbb{R}^{r \times k}$ is the first trainable low-rank matrix (down-projection)
- $B \in \mathbb{R}^{d \times r}$ is the second trainable low-rank matrix (up-projection)
- $r \ll \min(d, k)$ is the rank of the decomposition
- α is a scaling factor that controls the magnitude of the low-rank update

Only A and B are updated by the optimizer; W_0 remains frozen and requires no gradient storage or optimizer state. For a typical transformer with attention projections W_q, W_k, W_v, W_o and feed-forward projections $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$, LoRA adapters are attached to each projection, yielding a trainable parameter count that is a small fraction of the base model (on the order of tens of millions versus tens of billions).

At the data scales typical for domain-specific data-to-text tasks (on the order of several hundred to a few thousand labeled examples), LoRA's low-rank constraint also acts as a regularizer: by limiting the number of trainable parameters, it reduces the risk of overfitting and preserves the general linguistic capabilities acquired during pre-training [11].

QLoRA [6] extends LoRA with two memory-reduction techniques that together allow fine-tuning a 31-billion-parameter model on a single GPU with 40 GB of VRAM. First, the frozen base model weights are quantized to *4-bit NormalFloat* (NF4), a quantization datatype that is information-theoretically optimal for normally distributed weights. A *double quantization* step further quantizes the quantization constants themselves, reducing the per-parameter overhead. Second, *paged optimizers* leverage NVIDIA unified memory to transparently page optimizer state between GPU and CPU memory when the GPU reaches capacity, avoiding out-of-memory failures during gradient accumulation steps. The forward and backward passes are performed in `bfloat16` precision: the 4-bit weights are dequantized on the fly for each matrix multiplication, preserving numerical quality close to a full-precision fine-tune while keeping the memory footprint low.

3.3.2 Training Data Construction

Rather than collecting human-annotated data–triple pairs, the training data for the fine-tuned model is produced by *distilling* one of the pipeline described in Section 3.2. For each raw structured instance d_i processed by a tripler method, the pipeline already produces a reference text t_i and a set of normalized triples $T_i = \{(s_k, \pi_k, o_k)\}$. These two outputs, aligned by instance identifier, constitute exactly the joint target that the fine-tuned model should learn to produce in a single pass.

Each training example is formatted as a three-message conversation following the base model's chat template:

- **System prompt.** An instruction directing the model to convert one structured data instance into concise natural language and a corresponding set of RDF semantic triples (see Appendix B for the complete prompt text), and to return the result as a JSON object of schema `{"text": "...", "triples": [{"subject": "...", "predicate": "...", "object": "..."}]}`.
- **User message.** A prompt that presents the serialized instance and requests the reference text and triples (see Appendix B for the complete template). The wording and instance

serialization are kept byte-identical to the prompts used by the tripler pipeline at inference time, ensuring *prompt parity*: the token sequence seen during training matches the token sequence the model will encounter when served.

- **Assistant message (target).** A JSON string encoding the reference text and the normalized triples (see Appendix B for the expected output format): `{"text":` t_i`, "triples":` T_i`}`.

The messages are rendered through the base tokenizer's `chat_template` function, which applies the special tokens and formatting expected by the model architecture. This ensures that the token sequences fed to the trainer are identical to those assembled by the serving engine at inference time.

A notable consequence of this distillation approach is that the fine-tuned model is bounded by the quality of the reference labels: it can learn to reproduce the pipeline's outputs more efficiently and consistently, but it cannot exceed their quality. Improving the reference labels is an upstream concern addressed by the tripler pipeline design, not by fine-tuning.

The dataset is split into a training, evaluation and test set. The split is deterministic, controlled by a fixed random seed, so that results are reproducible across runs. One fine-tuned model is produced per domain, keeping the predicate vocabulary and verbalization style specialized to that domain.

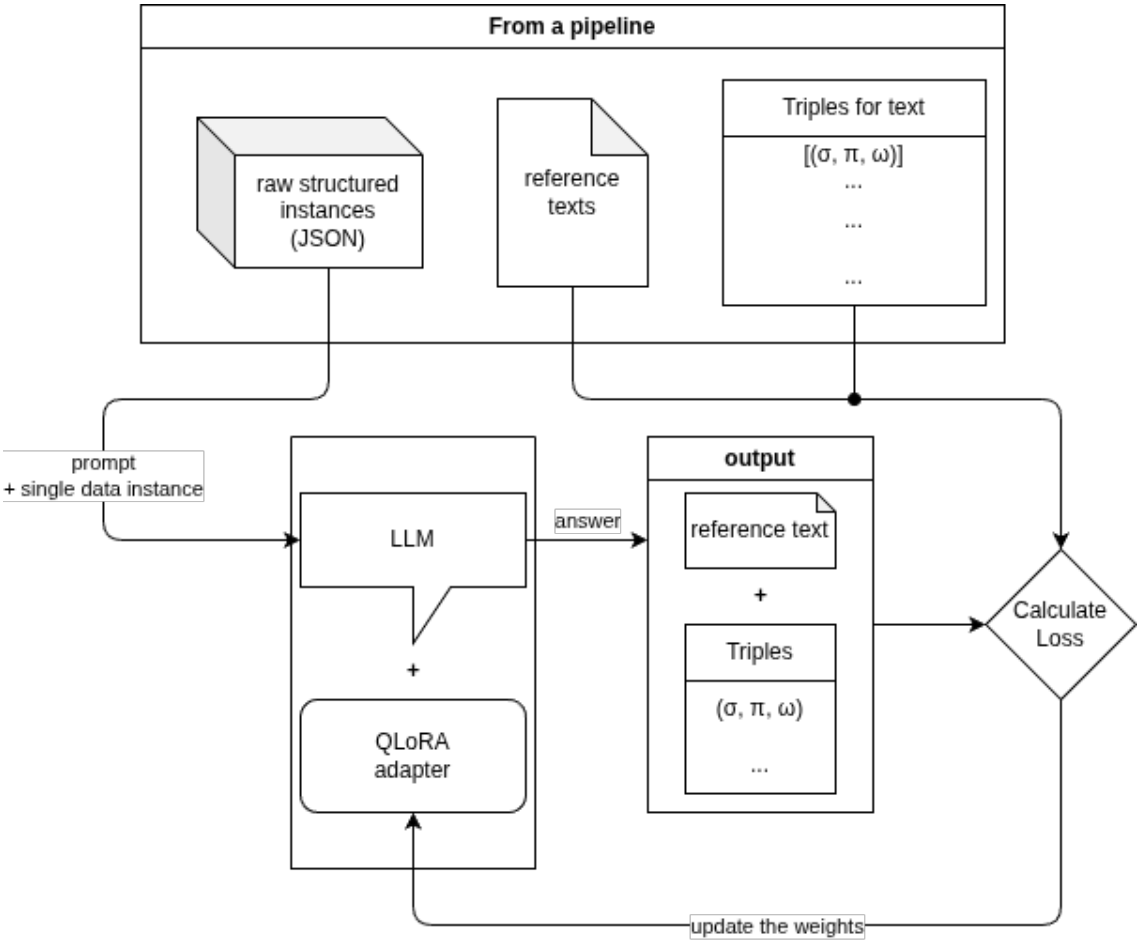

Figure 3.2: Fine-tuning flow: the raw structured instances, reference texts and triples are used to train a domain-specialized model via QLoRA. The resulting model can produce both outputs in a single decode pass.

Chapter 4

Surface realization — evolving programs using AlphaEvolve

4.1 Method Overview

The previous chapter considered the problem of meaning representation. Complex structured data was converted into semantic triples so that the information could be expressed in a compact, standardized, and machine-interpretable form. The problem considered in this chapter is the second stage of the data-to-text conversion: given the information contained in a set of triples, how should it be said in natural language?

This stage is called *surface realization*. Let $T = \{(s_i, p_i, o_i)\}_{i=1}^{n}$ denote a set of subject–predicate–object triples and let y denote a natural-language text. A surface realizer is a mapping

$$G : T \longrightarrow y, \tag{4.1}$$

which must express the facts in T fluently and coherently. The mapping is not limited to replacing predicates with fixed phrases. It can require entity ordering, aggregation of related facts, selection of referring expressions, realization of numerical values, and the choice of a suitable sentence structure. The generated text should preserve the information in the triples while avoiding unsupported statements and unnecessary repetition.

A large language model can be asked to perform this mapping directly using a zero-shot or few-shot prompt. Such a solution can produce fluent text, but the realization behavior is then distributed across model parameters, prompt instructions, and sampling decisions. It is consequently difficult to inspect which rules were applied, to reproduce a particular decision, or to diagnose a hallucination. A direct LLM call also makes the model part of every inference step. The approach taken here changes the role of the LLM: rather than asking it to verbalize each input instance directly, it is asked to construct a program that performs the conversion. The resulting program can be executed, tested, and inspected as an explicit surface realizer.

This idea is directly motivated by the work of Lango and Dusek [17], who describe a neurosymbolic framework in which LLM agents implement an NLG system from scratch. Their agents produce interpretable rule-based Python generators from RDF triples, without requiring in-domain human reference texts or conventional supervised training. The generated code provides a transparent representation of the realization process and can generate text using ordinary CPU execution. The work demonstrates that LLMs can be used to build an NLG system rather than only to act as the NLG system at inference time.

The distinction in this work concerns how the generator program is obtained. Lango and Dusek use collaborative interactions between multiple LLM agents to design and refine the rule-based gen-

erator. Here, Google's AlphaEvolve idea is used instead: the LLM is placed inside an evolutionary optimization loop and proposes mutations of an existing program. The implementation used in the experiments is OpenEvolve, an open-source evolutionary coding framework that provides the AlphaEvolve-style orchestration layer.

During evolution, an LLM receives the current program together with selected alternative programs and task-specific instructions. It then proposes a modified program, which is evaluated on RDF-to-text examples. The evaluation compares generated texts with references using automatic metrics and can additionally use an LLM-based judge. The evolutionary database stores candidate programs and their measurements, making the final result not merely a generated response, but an executable and inspectable artifact.

The remainder of this chapter first introduces agentic programming and the AlphaEvolve framework. It then describes OpenEvolve, the program representation and database, candidate selection, prompt construction, LLM response handling, and the evaluation process used for surface realization.

4.2 Modern approaches for writing complex programs

Writing a complex program is more than translating a short natural-language description into a code fragment. A complete solution usually requires several decisions about decomposition, data structures, interfaces, error handling, testing, and the interaction between multiple components. These decisions are often coupled: a change to one component can invalidate assumptions made by another component. Consequently, the generation of a single locally plausible code fragment does not guarantee that a complete program is correct or useful.

Traditional program synthesis and code-completion systems address only part of this problem. Program synthesis generally searches for a program satisfying a formal specification or a collection of input–output examples, whereas code completion systems predict a local continuation of an existing source file. Both approaches can be highly effective for constrained tasks, but neither usually performs the complete iterative workflow required for a large software engineering task. Such a workflow includes requirement analysis, implementation, execution, diagnosis of failures, modification of several files, and repeated validation.

4.2.1 Agentic programming

The recent development of large language models has led to a transition from one-shot code generation towards *agentic programming*. Wang et al. [31] define this paradigm through the use of LLM-based agents that autonomously plan, execute, and refine software-development tasks while interacting with external tools. The LLM is therefore embedded in a control loop rather than being used only as a function that maps a prompt to a final source fragment. The agent can inspect the workspace, formulate intermediate goals, edit files, run tests or compilers, interpret the resulting diagnostics, and decide what action should be taken next.

An agentic programming system can be characterized along several related dimensions. It can be reactive, responding to a user request or a tool result, or proactive, selecting subsequent actions in pursuit of a longer-term goal. It can operate in a single turn or across multiple turns, and it can be standalone or augmented with tools such as terminals, compilers, debuggers, profilers, language servers, and version-control systems. Finally, it can be static, using the same procedure for every task, or adaptive, changing its plan in response to observations from the development environment.

4.2.2 Current state of the art

The state of the art in agentic programming is represented not by a single model architecture, but by a combination of a capable code-oriented LLM and an orchestration layer. Modern systems can use long context windows, repository retrieval, structured tool calls, shell commands, test runners, and version control operations. Multi-agent designs can assign different roles to planning, implementation, testing, code review, and documentation. In all of these cases, the central improvement over one-shot generation is the ability to use concrete feedback from the environment to revise an earlier decision.

The most important enablers of these systems are structured prompting, state and context management, tool integration, and execution monitoring [31]. Structured prompts provide the model with a role, a task decomposition, and constraints on the output. Memory mechanisms preserve plans, tool results, and previous decisions that no longer fit into the immediate context window. Tool interfaces ground the model's actions in executable operations, while execution monitoring provides an observation on which the next action can be based.

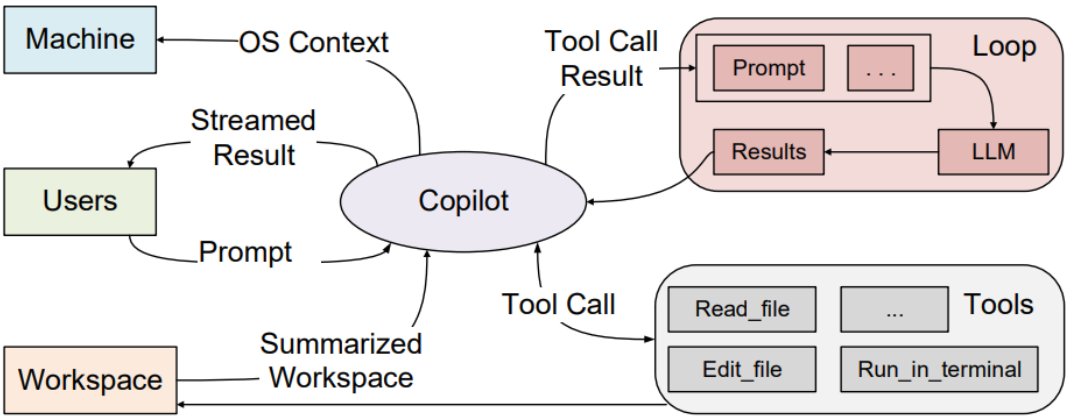

FIGURE 4.1: From Wang et al. [31] fig. 1: An example workflow of an AI coding agent.

4.2.3 Challenges and possible improvements

Despite the rapid progress, agentic programming remains limited in several ways. First, generated code is probabilistic and can contain syntactic errors, incorrect assumptions, hallucinated APIs, or changes that solve one test case while breaking another. A successful execution on a small test set is not a proof of correctness. Second, long software-engineering tasks produce more context than can be placed in a single prompt. Important decisions can be forgotten, and repeated failures can be rediscovered instead of being avoided. Third, conventional compilers and development tools expose diagnostics designed primarily for human interpretation. Their outputs do not always provide the structured causal information needed by an autonomous system to relate a failure to a particular edit.

Evaluation is another central limitation. Many existing benchmarks focus on small, self-contained tasks, a small number of programming languages, or a binary pass/fail signal. Such benchmarks do not fully measure long-horizon planning, tool-use efficiency, recovery from failure, repository-level changes, or collaboration with a human developer [31]. In addition, autonomous execution creates safety and privacy concerns: an agent may access sensitive files, execute unsafe

commands, or make changes that are difficult to inspect and reverse. Finally, repeated inference and tool calls can be expensive, especially when a large model is used for every intermediate step.

Several improvements follow from these limitations. Toolchains can expose structured diagnostics, abstract syntax trees, intermediate representations, profiling data, and explicit links between edits and failures. Context can be organized into short-term interaction history, medium-term plans, and long-term project memory, with retrieval and summarization used to control its size. Reversible edits, sandboxed execution, least-privilege tool access, and human approval for high-impact operations can improve safety. Benchmarks should also contain realistic multi-file tasks, multiple languages, tool interaction, intermediate feedback, and metrics for robustness and recovery rather than functional correctness alone.

4.3 AlphaEvolve

4.3.1 Motivation and problem formulation

AlphaEvolve is an evolutionary coding agent introduced by Novikov et al. [20]. It was designed for scientific and engineering problems in which candidate solutions can be represented as programs and assessed automatically. The motivation is that LLMs can propose creative algorithmic ideas, but an isolated LLM response is not a reliable optimization procedure. A generated program can be syntactically valid and still be slower, less accurate, or less robust than the program that preceded it. AlphaEvolve places the LLM inside an evolutionary loop in which every proposed change is grounded by execution and scoring.

This design solves a problem that is difficult for either component alone. Evolutionary search supplies selection, persistence, and a mechanism for backtracking from unsuccessful ideas, while the LLM supplies mutations that can change the program at a semantic level rather than only tuning a fixed vector of numerical parameters. The evaluator prevents an attractive explanation or a plausible-looking patch from being accepted without evidence of improved performance.

The examples reported by Novikov et al. [20] illustrate the breadth of this formulation. AlphaEvolve discovered a procedure for multiplying two 4×4 complex-valued matrices using 48 scalar multiplications, improving on the previously known 49-multiplication result in that setting. In engineering applications, it produced a matrix-multiplication kernel with an average speedup of 23% and contributed to a reported 1% reduction in Gemini's training time. These results are not required for using the framework, but they demonstrate why evolving executable algorithms can be more useful than generating a single static answer.

4.3.2 Architecture of AlphaEvolve

The description of AlphaEvolve separates the discovery process into the following cooperating components [20]:

1. The *task specification* supplies the initial program, the evaluator, optional configuration, and any fixed domain knowledge.

2. The *prompt sampler* selects previously evaluated programs and assembles the context shown to the language model.

3. The *LLM ensemble* proposes one or more modifications to a selected program. Models with different capabilities or latency can be combined so that many inexpensive ideas and occasional more capable ideas are explored.

4. The *mutation layer* parses the model response and applies the proposed changes to the current source code. For large programs, targeted diff blocks avoid sending or rewriting unrelated code.

5. The *evaluators* execute the candidate and return its metrics. Evaluation can include cascades, parallel test cases, multiple objectives, and additional LLM-generated feedback.

6. The *program database* stores candidates and metrics and controls the balance between exploiting high-scoring programs and exploring different program structures.

7. The *distributed controller* coordinates LLM requests and evaluations asynchronously so that slow evaluations do not prevent other candidates from being processed.

Let $\mathcal{P}$ denote the space of programs that satisfy the interface of a task. The user supplies an evaluator

$$h : \mathcal{P} \longrightarrow \mathbb{R}^m,$$

which returns one or more scalar metrics. The metrics are conventionally maximized, although the evaluator can negate a quantity such as execution time when minimization is required. Starting from an initial program p_0, the system repeatedly generates a candidate p', evaluates it using h, and stores the resulting program together with its metrics. The database then determines which previously evaluated candidates should influence later generations.

The interaction between these components is deliberately cyclic. A program is not selected only once at the beginning of the experiment. Its score, code, and observed failures become part of the context used for subsequent mutations. The database consequently acts as an external memory for the evolutionary agent.

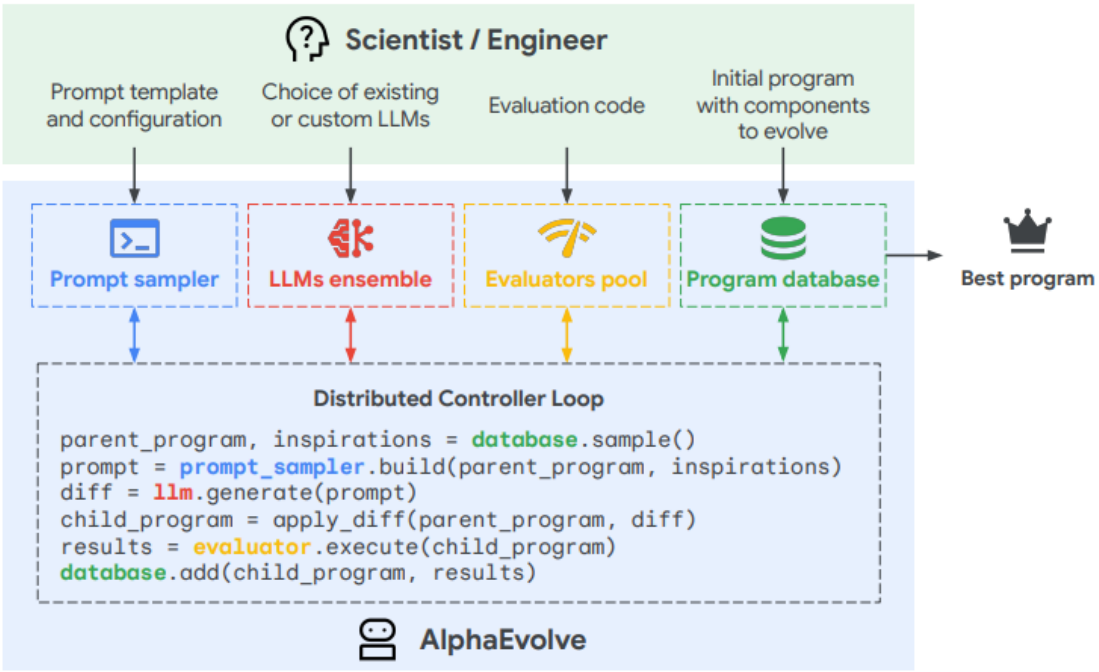

FIGURE 4.2: From Novikov et al [20] fig. 2: Expanded view of the AlphaEvolve discovery process.

4.3.3 Task specification and program representation

An AlphaEvolve task has three essential inputs. First, an initial program must be complete enough to run, even if its implementation is deliberately simple (the complete initial program is reproduced in Appendix C). Second, the part of the source code that may be changed is marked with `# EVOLVE-BLOCK-START` and `# EVOLVE-BLOCK-END`. The remaining code forms a fixed skeleton that defines imports, data types, and the public interface. Third, an evaluator function must be provided. The evaluator receives the generated program, executes it on the task-specific inputs, and returns a dictionary of scalar metrics.

4.3.4 Prompt sampling and creative generation

AlphaEvolve prompts contain more than the current program. Depending on the configuration, the context can contain high-scoring programs, structurally diverse programs, previous attempts, evaluator outputs, artifacts, explicit human-written instructions, and relevant examples or literature. The complete experiment-specific prompt composition is shown in Appendix A.1. This context allows an LLM to compare alternatives rather than repeatedly proposing a mutation without knowing what has already been tried.

The LLM is asked to make a concrete source-level change. For a diff-based mutation, the expected response is a sequence of blocks of the form

```
<<<<<< SEARCH
original code
=======
replacement code
>>>>>> REPLACE
```

The search part identifies the source segment to be replaced, and the replace part contains the candidate implementation. The complete mutation template is provided in Appendix A.1.2. A complete rewrite can be used for short programs or when a targeted change is not appropriate. In both cases, the model is instructed to preserve the public interface and to optimize the metrics defined by the evaluator.

AlphaEvolve uses an ensemble rather than relying on only one generation model. In the original system, a faster model increases the number of explored ideas, while a more capable model can supply less frequent but more substantial changes [20]. The ensemble is not an ensemble in the sense of averaging source code. A single model response is selected for each mutation, after which the resulting program is evaluated independently.

4.3.5 Evaluation, evolutionary storage, and distributed execution

AlphaEvolve supports several mechanisms for making the evaluation stage both useful and affordable. An evaluation cascade can apply inexpensive tests first and run expensive tests only for candidates that pass earlier thresholds. Parallel test cases can be distributed across workers. Multiple metrics can be returned simultaneously, both because they may represent genuine objectives and because programs that perform well under different metrics can provide more varied context for future mutations. Optional LLM feedback can be used for properties that are difficult to express in a conventional evaluator, such as readability or simplicity.

The system is implemented as an asynchronous pipeline. LLM generation and evaluation are started whenever their required inputs are available, and the controller waits only at dependencies

between stages. This maximizes the number of candidates evaluated within a fixed computation budget instead of minimizing the latency of an individual candidate. The resulting evolutionary database is inspired by MAP-Elites and island-based population models. MAP-Elites preserves high-performing programs in different regions of a feature space, while islands allow partially independent populations to explore different directions before selected migration.

4.3.6 Scope and limitations of AlphaEvolve

The unique feature of AlphaEvolve is the combination of open-ended LLM mutations with machine-gradeable selection. It is not a general replacement for a software engineer, and it cannot be applied directly when candidate quality cannot be measured by an evaluator. Its results depend on the quality, coverage, and resistance to exploitation of the evaluation function. A weak metric can reward a program that optimizes the measurement rather than the intended task. The search can also require substantial computation because many candidates must be generated and evaluated, and there is no general guarantee that the best program will be found within a finite budget.

These limitations are particularly relevant to surface realization. Text quality is only partially captured by lexical metrics, while LLM judges may be inconsistent or sensitive to output formatting. The experiment therefore retains both reference-based metrics and judge feedback, logs failed cases, and keeps the mutation interface narrow enough that candidates can be evaluated repeatedly.

4.4 OpenEvolve

4.4.1 Open-source implementation

OpenEvolve is an open-source implementation of the AlphaEvolve design [27] and is used as the execution framework for all surface-realization experiments. The implementation exposes the same basic interface as AlphaEvolve: an initial program, an evaluator, and a configuration determine an iterative process in which programs are proposed, scored, and registered for future use.

The main controller loads the initial source, initializes the LLM ensemble, prompt sampler, evaluator, and program database. The initial program is evaluated before the evolutionary workers are started. On a fresh run it is then inserted into the database as generation zero. This makes the baseline available to the selection mechanism and ensures that evolution starts from a measured candidate rather than an unscored source file.

4.4.2 Execution architecture used in the thesis

The local implementation uses parallel workers for process-based parallelism. Before a worker is submitted, the controller selects a parent program and inspiration programs, which are selected the same way as the parent program, and are used as a source of inspiration for the LLM. Then the controller builds the prompt, calls the LLM, parses the response, and evaluates the resulting source.

Consequently, the logical evolution loop is:

1. select a parent and inspirations;
2. assemble a prompt from the selected context;
3. generate and parse an LLM mutation;

4. execute the candidate evaluator;
5. register the candidate, metrics, and feedback;
6. repeat until the iteration budget, target score, or stopping rule is reached.

The implementation is asynchronous in practice. Several workers can use snapshots of the program database made close together, and their results can be incorporated in a different order from the numerical iteration identifiers. This increases throughput but means that a candidate does not necessarily use the immediately preceding candidate as its parent.

4.4.3 Extensions made for the experiments

Several changes were made to the OpenEvolve's algorithm. They address both the search distribution and the failure modes that are common when a language model is required to emit an exact source-code patch.

Fitness-weighted and island-specific selection. The database was extended with roulette-wheel selection. Parent and inspiration programs can be sampled with probabilities derived from their fitness instead of being selected uniformly. A small positive lower bound is used in the fitness weights so that low-scoring programs are not removed from consideration immediately. LLM configurations can also be assigned to islands, allowing a population to be associated with a particular model or model mixture. An optional iteration-based model schedule was added for periodically forcing a chosen model.

Configurable inspirations. The number of inspiration programs that are selected and added to the prompt can be controlled.

More robust SEARCH/REPLACE application. The original exact line matching was extended with trailing-whitespace cleanup and common-indentation handling. A model can therefore return a source block with a different outer indentation while the replacement is still inserted at the correct nesting level. If the response has no parsable diff, a corrective prompt asks the LLM to return the required format. If a syntactically valid diff does not change the source because its SEARCH block does not match, a second corrective prompt includes the parent code and asks the LLM to repair the SEARCH section. The exact repair prompts are given in Appendix A.1.5.

Retention of failed candidates. Malformed or unchanged proposals are written to a separate directory instead of being silently discarded. Duplicate source code is detected before insertion. These changes prevent repeated evaluation of identical candidates and preserve evidence needed to diagnose the behavior of the LLM and the mutation parser.

Explicit zero scores for timeouts. The evaluator integration was changed so that timeout and failed-evaluation paths give a score of 0.

4.5 Program database and initial program

4.5.1 Program records and persistence

The program database is implemented in a way to store those principal fields:

- a unique identifier and the program source;
- the identifier of the parent program, generation number, timestamp, and iteration at which the program was found;
- a dictionary of evaluation metrics, including the fitness metric;
- derived values such as complexity and diversity and metadata such as island assignment and a human-readable change summary;
- optional prompts and LLM responses associated with its creation;
- optional artifacts and embeddings used for feedback and novelty checks.

The active database is maintained in memory for fast selection. For each program, feature coordinates are calculated before insertion. In the surface-realization task, code length is used as the built-in complexity feature, while diversity is approximated from source-code length, line-count, and character-set differences. Feature values are scaled and assigned to bins. The feature coordinates form a cell key, for example a pair of coordinates for complexity and diversity.

At most one program is retained as the occupant of a feature cell. If a new program maps to an empty cell, the cell is occupied. If it maps to an occupied cell, the program with the better fitness is retained as the cell occupant. The database separately stores a bounded set of elite programs. The absolute best program is tracked independently so that it is protected from ordinary population cleanup. When the configured population limit is exceeded, the lowest-fitness unprotected programs are removed from the active in-memory population, together with their references in islands and cells. The main limits are configurable.

4.5.2 Initial program

The initial program, reproduced in Appendix C, defines a small `Triple` data class with `subject`, `predicate`, and `object` fields. Its evolution block contains the following baseline behavior: `predict` receives a list of triples and returns a sentence describing the predicate, subject, and object of the first triple. This implementation is intentionally minimal. It is syntactically valid and satisfies the interface, but it ignores additional triples, treats every predicate identically, and does not model relations between entities.

The purpose of evolution is not merely to make this baseline longer. The desired search direction is from a generic fallback sentence towards a domain-aware surface realizer. Useful changes include mapping predicates to natural-language templates, grouping triples by subject, identifying a main entity, ordering attributes, joining related entities, selecting relative clauses, and maintaining grammatical agreement. These changes are evaluated by the generated text rather than being prescribed as a fixed hand-written algorithm, which allows multiple realization strategies to be explored.

4.6 Program selection and prompt engineering

4.6.1 Parent and inspiration selection

Selection is performed before a worker process is launched. For roulette selection, the parent is selected from the island using a fitness-weighted softmax distribution. The use of a distribution rather than always selecting the best program retains a non-zero probability of returning to less explored solutions.

Inspirations are selected separately from the parent. They are alternative programs that can provide a design idea without being the source that is directly modified. With roulette, their probabilities are also based on fitness, and the parent itself is excluded from the candidate inspiration set. The selected parent and inspirations are passed to the worker, which reconstructs their source code from the database.

4.6.2 Prompt composition

The system message used for the surface-realization experiment is reproduced in Appendix A.1.1. It contains four kinds of fixed information:

- the role of the model as an expert data engineer;
- the domain and the required `predict` function signature;
- an example in which several triples are combined into one complex sentence;
- the list of predicates that may occur, with one example triple for each predicate.

The user message is assembled for each selected parent. Its complete template and runtime components are shown in Appendix A.1.2. Its main parts are:

- the current metrics and fitness score;
- optional evaluator artifacts, such as low-scoring triples and the generated text that caused them (see Appendix A.1.4);
- the complete current source code;
- the mutation task, including the requirement to improve the score, preserve the interface, and use the exact SEARCH/REPLACE format.

Consequently, the code and score of the selected parent are shown directly in the current-program section. An inspiration is rendered separately with its own score and source code. A high-scoring program can therefore provide a strong implementation pattern, while a structurally different inspiration can suggest a different way of grouping or ordering triples.

4.6.3 SEARCH/REPLACE instructions and their extensions

The mutation prompt asks for a targeted edit rather than a complete rewritten file. The SEARCH section must identify source text present in the current program, and the REPLACE section must contain the proposed replacement. The controller extracts all matching blocks with a regular expression, applies them line by line, and checks whether the resulting source differs from the parent. This format makes changes auditable and reduces the risk that the LLM rewrites fixed parts of the program unintentionally. The full mutation prompt is given in Appendix A.1.2.

The format is also a significant source of failure. A model can describe a patch in prose, omit one of the delimiters, copy an outdated version of the source, or use an indentation that prevents an exact match. These problems are adressed in two ways. First, trailing whitespace is normalized and common indentation is removed while matching, after which the replacement is re-indented at the original location. Second, malformed responses and unapplied responses are sent to dedicated repair prompts. These safeguards preserve the replacement text where possible and focus the subsequent LLM call on producing a matchable SEARCH section; the repair messages are listed in Appendix A.1.5.

4.7 Large Language Model

4.7.1 Parsing the LLM response

OpenEvolve uses an OpenAI-compatible client interface, so the same orchestration code can communicate with hosted models or local servers. The response is processed in the worker in the following order:

1. The response is searched for one or more SEARCH/REPLACE blocks.
2. Each block is normalized into source lines and applied to the parent source.
3. If at least one block is found but no source line is changed, the response is treated as an unapplied mutation rather than as a successful no-op.
4. A new identifier is assigned and the candidate is passed to the evaluator.

When no valid block is found, the response is not immediately treated as a failed iteration. Up to four attempts are made. The first repair instruction explains the required delimiter format. If a block was parsed but could not be applied, the repair instruction additionally contains the original parent code and asks the model to correct the SEARCH section while preserving the replacement. The intermediate invalid responses are joined and retained as diagnostic feedback.

If all repair attempts fail, a child record carrying the unchanged parent code is returned with an error and a `no_valid_diffs` or `no_changes` artifact. The controller writes this record to the list of broken programs and does not add it to the active evolutionary population.

OpenEvolve also contains a full-rewrite mode. When diff-based evolution is disabled, a fenced code block or the plain response is parsed as the complete candidate source. This mode was retained for compatibility, but the surface realization experiments use targeted diffs because the evolved function is embedded in a fixed program skeleton.

4.7.2 Invalid programs and execution safety

A generated program can fail before it produces any text. Dynamic loading can fail because of a syntax error or missing import; the required `predict` function can be absent or have the wrong signature; and a call can return a non-string value, raise an exception, or run indefinitely. These cases are converted into zero-valued fitness and artifacts describing the failure rather than terminating the entire evolution run. The artifacts can then be included in a later prompt, where they provide a concrete example of what should be avoided.

4.8 Evaluation

4.8.1 Evaluator integration

The OpenEvolve `Evaluator` is a generic wrapper around the task-specific evaluation file. For every candidate, it invokes the function named `evaluate`. The result contain metrics and artifacts. The wrapper applies the configured timeout, handles retries, converts both result formats to a common representation, and collects pending artifacts for the program record.

The initial program is passed through the same evaluator before evolution starts. This establishes the baseline against which later candidates are compared. A worker follows the same path after it has applied an LLM mutation. The task-specific evaluator is therefore the boundary

between source-code generation and evolutionary selection: OpenEvolve does not infer whether a sentence is good from the source code itself.

4.8.2 Reference data and candidate execution

The surface-realization evaluator loads a supplied reference data corpus. Each selected entry contains a triple set and one or more reference lexicalizations.

Before the examples are processed, the evaluator loads the candidate module dynamically and checks that `predict` exists and accepts exactly one argument. For each reference data entry, the list of triples is passed to `predict`. A result is accepted only when it is a string. An empty string receives a per-example zero lexical score. Exceptions, invalid return types, and timeouts terminate the affected evaluation path and produce zero-valued metrics with an explanation in the artifacts.

4.8.3 Reference-based scoring

For every non-empty generated text, the evaluator computes a BLEU score against all reference lexicalizations associated with the entry. The average over the selected domain entries is returned as an aggregate reference-based measure.

BLEU value is used as `combined_score`, and it is the quantity used by the evolutionary database for fitness comparisons.

4.8.4 LLM as a judge

Themis is a language model dedicated to the evaluation of natural language generation, rather than a metric based on surface-form overlap. Hu et al. [12] propose Themis as an 8-billion-parameter evaluator trained using the NLG-Eval corpus, which combines human and GPT-4 annotations from multiple NLG tasks. The model was designed to address several limitations of conventional metrics and generic prompted judges. In particular, it can evaluate an output without a reference text, accept task-specific evaluation aspects and criteria, and return both a rating and an analysis explaining that rating. The latter property makes the evaluation useful not only for ranking outputs, but also for identifying which aspects of a generated text require improvement.

Here an LLM is used as a judge for the surface realizer. For each non-empty generated text, the evaluator sends the input triples and the text to the configured judge, without sending the reference texts to that judge. The evaluation criterion requires the text to express the triples accurately, to avoid information not present in the triples, and, where possible, to use a single complex sentence instead of several disconnected sentences. The exact judge prompt is reproduced in Appendix A.2. The judge first returns a concise review and then a rating on a scale from 1 to 5. The ratings are averaged over the evaluation examples and divided by 5. When the judge is enabled, this normalized value is used as the combined evolutionary fitness, while BLEU remains available as a reference-based diagnostic metric.

The evaluator does not depend on a particular set of Themis model weights. The name, endpoint, and credentials of the judge are supplied through the configuration. Therefore, the same evaluation role can be assigned to another suitable LLM when required. The term “Themis” in the experiment configuration consequently identifies the default judge setup, while the evaluation protocol can also be used with an alternative model. This allows the judge to favor a faithful and compact realization even when it uses wording that has limited n-gram overlap with the references.

4.8.5 Artifacts as evaluator feedback

The evaluator returns low-scoring examples as artifacts. An LLM-judge artifact contains the input triples, the generated text, the judge review, and the score. If too many examples qualify, at most three are sampled for storage and prompt rendering (the message format is shown in Appendix A.1.4). When the LLM judge is disabled, low BLEU examples contain the references as well. On the next mutation, these artifacts are rendered under the feedback section of the user prompt. The resulting feedback loop is therefore more specific than a scalar fitness value: the LLM can see which triples were omitted, which relation was expressed poorly, or which output was split into several sentences.

The evaluator distinguishes metrics from artifacts. Metrics are numeric values used by the database, whereas diagnostic strings, reviews, traces, and error messages are not included in the fitness calculation. This separation prevents an explanatory text from accidentally changing the ranking while still making the explanation available to the next generation.

4.8.6 Registration and continuation of the loop

After evaluation, the worker creates a child `Program` containing the candidate source, parent identifier, generation, iteration, metrics, and a change summary. The database then calculates feature coordinates, updates the corresponding MAP-Elites cell and the archive, enforces the population limit, and updates the global and island-specific best-program references. Artifacts and prompts are stored, and the evolution tracer records the parent, child, metrics, source code, response, and improvement delta when tracing is enabled. A new iteration is then submitted, so the evaluated child can influence later parent and inspiration selection.

This ordering is the central feedback mechanism of the method. A score without registration would not affect the search, and a database without evaluation would have no objective basis for selection. The repeated interaction between the two is what converts LLM code generation into an evolutionary surface realization procedure.

Chapter 5

Experiments

5.1 Meaning representation

5.1.1 Data Collection

The experimental evaluation for meaning representation requires structured data from diverse domains. To obtain such data, *quintd* [13] is used, a framework for collecting *ad hoc* data-to-text datasets. The purpose of this type of collection is to provide data that is not part of the training data of the evaluated language models. Quintd selects examples using a random seed and can also obtain up-to-date information from external data sources.

The source collection used in the experiments is denoted by S_0. It contains the five domains available in the corresponding quintd collection: GSMArena, OpenWeather, Our World in Data (OWID), Wikidata, and ice hockey. The ice-hockey data is retained as part of the source collection for completeness, but it is not used in the experiments described below. The pipeline comparison and the fine-tuning experiments use the same four-domain subset consisting of GSMArena, OpenWeather, OWID, and Wikidata. This distinction is important because the ice-hockey domain is present in the quintd collection but is absent from the later S_1, S_2, S_3 and S_4 datasets.

The four experimental datasets are:

GSMArena Mobile phone specifications scraped from the GSMArena website. Each instance describes a single phone model with nested categories such as Network, Launch, Body, Display, Platform, Memory, Camera, and Battery, containing attribute–value pairs.

OpenWeather Five-day weather forecasts obtained from the OpenWeather API. Each instance contains a time series of 40 three-hour intervals, with each interval recording temperature, humidity, wind speed, cloud coverage, and weather conditions.

OWID Country-level time-series metrics from the Our World in Data project. Each instance represents a single metric, such as life expectancy, vaccination rates, or reproduction rate, for a specific country and contains yearly data points spanning several decades.

Wikidata Knowledge-graph entities and their properties from Wikidata. Each instance describes an entity, such as a country, film, or organization, with a list of property–value pairs such as capital, cast member, or language used.

The fifth domain, *ice hockey*, is therefore not included in the pipeline comparison, the human evaluation, or the fine-tuning experiments. Its presence in the original S_0 collection should not be interpreted as evidence that it was used in the later four-domain experiments.

All experimental datasets are provided as JSON files containing arrays of structured records. A *raw structured instance* is defined as a single JSON object representing one self-contained data record from one of the four experimental domains. Each instance contains nested fields with structured data—time series, categorical values, numerical measurements, or hierarchical attribute lists—that must be converted into semantic triples. Let $D = \{d_1, \ldots, d_N\}$ denote the set of N such instances extracted from the input JSON for a given domain.

Large language model. All LLM invocations across the three conversion methods use *Gemma 4 31B IT* [28, 29], a 31-billion-parameter instruction-tuned language model released by Google. The model is served via *vLLM* [16], a high-throughput inference engine that uses *PagedAttention* to manage key–value cache memory and enable batched inference with dynamic sequence lengths.

5.1.2 Experimental setup for pipelines

The pipeline comparison uses 100 instances from each of the four experimental domains. Each of the three pipelines is run on the same domain-specific instances, resulting in twelve domain–pipeline experiments. The three pipelines are Text-First Extraction with Normalization, Blueprint-Iterative Refinement, and Evolving Catalog.

The generated outputs are evaluated using an LLM as a judge. The judge is the same Gemma 4 31B IT model, but it is used in a separate evaluation role rather than as part of the extraction pipeline. Two transitions are assessed independently:

1. the conversion from the raw structured instance to the generated natural-language reference text; and

2. the conversion from the reference text to the generated semantic triples.

Each transition is evaluated according to four criteria: summary, completeness, faithfulness, and omissions. The complete criterion prompts are provided in Appendix A.2.1. Summary measures whether the output is concise, coherent, non-redundant, and preserves the source's core message. Completeness measures coverage of the source's main points and critical facts, constraints, context, entities, relationships, values, and conclusions. Faithfulness measures whether the claims in the output are supported by the source and preserve its meaning, penalizing hallucinations, contradictions, and misinterpretations. Omissions measures whether omitted content removes critical context, changes the meaning, hides an important conclusion or constraint, or creates a misleading imbalance. Every criterion is scored on an ordinal scale from 1 to 5, where 5 represents the best result. Each criterion uses a separate judge prompt. The four criteria are separated so that a missing fact is not treated as the same type of error as an unsupported claim. Since the evaluation contains 400 instances, two transitions, and four criteria, it requires 3,200 judge calls. For each domain and pipeline, the score for each criterion is averaged over the 100 instances. The resulting averages are used to compare the three conversion methods.

The evaluation output also contains descriptive statistics. These statistics are computed without additional LLM calls and describe the size and structure of the generated outputs: the number of primitive JSON elements, reference words, reference sentences, reference subsentences, extracted triples, and unique predicates. The overall text and triples scores are calculated as the mean of the four criteria belonging to the corresponding transition.

Table 5.1: Data-to-text judge scores for Blueprint-Iterative Refinement.

domain	summary	completeness	faithfulness	omissions	overall
GSMArena	0	0	0	0	0
OpenWeather	0	0	0	0	0
OWID	0	0	0	0	0
Wikidata	0	0	0	0	0
Average	0	0	0	0	0

Table 5.2: Text-to-triples judge scores for Blueprint-Iterative Refinement.

domain	summary	completeness	faithfulness	omissions	overall
GSMArena	0	0	0	0	0
OpenWeather	0	0	0	0	0
OWID	0	0	0	0	0
Wikidata	0	0	0	0	0
Average	0	0	0	0	0

Table 5.3: Additional statistics for Blueprint-Iterative Refinement.

domain	json_elements	ref_words	ref_sentences	ref_subsentences	num_triples	unique_predicates
GSMArena	0	0	0	0	0	0
OpenWeather	0	0	0	0	0	0
OWID	0	0	0	0	0	0
Wikidata	0	0	0	0	0	0
Average	0	0	0	0	0	0

Table 5.4: Data-to-text judge scores for Text-First Extraction with Normalization.

domain	summary	completeness	faithfulness	omissions	overall
GSMArena	0	0	0	0	0
OpenWeather	0	0	0	0	0
OWID	0	0	0	0	0
Wikidata	0	0	0	0	0
Average	0	0	0	0	0

Table 5.5: Text-to-triples judge scores for Text-First Extraction with Normalization.

domain	summary	completeness	faithfulness	omissions	overall
GSMArena	0	0	0	0	0
OpenWeather	0	0	0	0	0
OWID	0	0	0	0	0
Wikidata	0	0	0	0	0
Average	0	0	0	0	0

Table 5.6: Additional statistics for Text-First Extraction with Normalization.

domain	json_elements	ref_words	ref_sentences	ref_subsentences	num_triples	unique_predicates
GSMArena	0	0	0	0	0	0
OpenWeather	0	0	0	0	0	0
OWID	0	0	0	0	0	0
Wikidata	0	0	0	0	0	0
Average	0	0	0	0	0	0

Table 5.7: Data-to-text judge scores for Evolving Catalog.

domain	summary	completeness	faithfulness	omissions	overall
GSMArena	0	0	0	0	0
OpenWeather	0	0	0	0	0
OWID	0	0	0	0	0
Wikidata	0	0	0	0	0
Average	0	0	0	0	0

Table 5.8: Text-to-triples judge scores for Evolving Catalog.

domain	summary	completeness	faithfulness	omissions	overall
GSMArena	0	0	0	0	0
OpenWeather	0	0	0	0	0
OWID	0	0	0	0	0
Wikidata	0	0	0	0	0
Average	0	0	0	0	0

Table 5.9: Additional statistics for Evolving Catalog.

domain	json_elements	ref_words	ref_sentences	ref_subsentences	num_triples	unique_predicates
GSMArena	0	0	0	0	0	0
OpenWeather	0	0	0	0	0	0
OWID	0	0	0	0	0	0
Wikidata	0	0	0	0	0	0
Average	0	0	0	0	0	0

5.1.3 Human evaluation for pipelines

The human evaluation uses the same four criteria as the LLM-as-a-judge evaluation: summary, completeness, faithfulness, and omissions. It is limited to the transition from the raw structured instance to the semantic triples. For each of the four domains, ten instances are selected for each of the three pipelines. The evaluation therefore contains $4 \times 10 \times 3 = 120$ manually assessed instances. The scores are assigned by the author of the thesis using the same 1–5 scale as the automated evaluation. Domain-level scores and an across-domain average are reported for each pipeline.

Table 5.10: Human evaluation for Blueprint-Iterative Refinement.

Domain	Summary	Completeness	Faithfulness	Omissions
GSMArena	0	0	0	0
OpenWeather	0	0	0	0
OWID	0	0	0	0
Wikidata	0	0	0	0
Average	0	0	0	0

Table 5.11: Human evaluation for Text-First Extraction with Normalization.

Domain	Summary	Completeness	Faithfulness	Omissions
GSMArena	0	0	0	0
OpenWeather	0	0	0	0
OWID	0	0	0	0
Wikidata	0	0	0	0
Average	0	0	0	0

Table 5.12: Human evaluation for Evolving Catalog.

Domain	Summary	Completeness	Faithfulness	Omissions
GSMArena	0	0	0	0
OpenWeather	0	0	0	0
OWID	0	0	0	0
Wikidata	0	0	0	0
Average	0	0	0	0

5.1.4 Correlation between LLM and human evaluation

The agreement between the LLM judge and the human evaluator is computed from the paired 1–5 scores. There are four complementary statistics: Pearson's product–moment correlation, Spearman's rank correlation, Kendall's tau-b, and quadratic weighted Cohen's kappa. Pearson's coefficient measures linear association, while Spearman's coefficient and Kendall's tau measure rank-based association. Quadratic weighted kappa measures chance-corrected agreement and penalizes larger disagreements more strongly than disagreements between adjacent score categories [4].

The metrics are computed separately for each of the four criteria and for the two evaluation transitions. Two additional overall variants are available for each transition: a pooled variant that concatenates the four criteria and a per-instance-mean variant that first averages the four criteria for each instance. The two transitions are never pooled together.

Table 5.13: Correlation between LLM-as-a-judge and human scores.

Pipeline	**Domain**	N	**Pearson** r	**Spearman** ρ	**Kendall** τ	**QW-**κ
Blueprint-Iterative Refinement	all	0	0	0	0	0
Text-First Extraction with Normalization	all	0	0	0	0	0
Evolving Catalog	all	0	0	0	0	0

5.1.5 Experimental setup for fine-tuning

The fine-tuning experiments are conducted on the same four domains as the pipeline comparison: GSMArena, OpenWeather, OWID, and Wikidata. The training-and-validation collection is downloaded using seed S_1, while the held-out test collection is downloaded independently using seed S_2.

The raw instances from the S_1 collection are converted into reference text and semantic triples using the Text-First Extraction with Normalization pipeline. The aligned text–triple outputs are then used as the targets for a domain-specific model. For each domain, the resulting examples are split into 80% training data and 20% validation data. The S_2 collection remains separate and is used only for the final evaluation of the trained models. This split prevents the test instances from influencing either model fitting or training-configuration selection.

The fine-tuned model receives one structured instance and is trained to return both a natural-language reference and its semantic triples in one JSON output. The base model is Gemma 4 31B IT, and the adaptation is performed using QLoRA. The base weights remain frozen, while low-rank adapters are trained in the language-model attention and feed-forward projections. After training, the adapters are merged into the base model before evaluation.

5.1.6 Training Configuration

The fine-tuning comparison contains five configurations. The standard baseline uses three epochs. The additional *baseline-epoch1* variant keeps the baseline learning rate, adapter capacity, dropout, and weight decay, but trains for one epoch. The remaining variants change the learning rate, adapter capacity, or regularization in order to test their influence on the domain-specialized models.

All training is performed on NVIDIA A100-SXM4 GPUs with 40 GB of VRAM, using the PLGrid Athena cluster. The common effective batch size is 16, obtained from a per-device batch size of 1 and 16 gradient-accumulation steps. The optimizer is 8-bit paged AdamW, the learning-rate schedule is cosine, and the compute precision is bfloat16. The base model is loaded using 4-bit NF4 quantization with double quantization.

Table 5.14: QLoRA configurations used for fine-tuning.

Configuration	Epochs	LR	Warmup	r	α	Dropout	Weight decay
baseline	3	10^{-4}	0.03	16	32	0.05	0.00
baseline-epoch1	1	10^{-4}	0.03	16	32	0.05	0.00
low_lr	5	3×10^{-5}	0.05	16	32	0.05	0.01
higher_capacity	3	5×10^{-5}	0.05	32	64	0.05	0.01
regularized_capacity	5	3×10^{-5}	0.10	32	64	0.10	0.01

The LoRA configuration targets all attention and MLP projection matrices in the language-model backbone (`q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`). A positively scoped module pattern excludes the vision and audio towers of the multimodal checkpoint. This selects 420 target modules and approximately 25 million trainable parameters, which is less than 0.1% of the base model.

The maximum sequence length is adjusted per domain to accommodate the varying sizes of the raw structured instances while staying within the GPU memory budget. Wikidata uses 8192 tokens, GSMArena and OWID use 4096 tokens, and OpenWeather uses 6144 tokens. Instances exceeding the selected maximum are truncated.

After training, the LoRA adapter weights are combined with the frozen base model weights to produce a standalone model. This process, known as weight merging, computes $W_{\text{final}} = W_0 + \Delta W$

for each adapted layer. The merged model can be served by the same vLLM infrastructure as the base model and produces both a natural-language verbalization and the corresponding semantic triples in a single decode pass.

5.1.7 Evaluation of fine-tuned models

The fine-tuned models are evaluated on the held-out S_2 test sets for each domain. The evaluation compares the generated text with the reference text using BLEU and METEOR. The generated and reference triples are compared as sets of subject–predicate–object tuples. Micro-averaged triple precision, recall, and F1 are computed from the resulting true-positive, false-positive, and false-negative counts. Predicate adherence measures the proportion of generated predicates that belong to the predicate catalog used to construct the targets. A parse rate is recorded by the evaluation script as a diagnostic, but the principal result tables follow the requested metric set.

The results are reported separately for the five configurations.

Table 5.15: Fine-tuned model results for the baseline configuration.

Domain	Text BLEU	Text METEOR	Triple precision	Triple recall	Triple F1	Predicate adherence
GSMArena	0	0	0	0	0	0
OpenWeather	0	0	0	0	0	0
OWID	0	0	0	0	0	0
Wikidata	0	0	0	0	0	0
Average	0	0	0	0	0	0

Table 5.16: Fine-tuned model results for the baseline-epoch1 configuration.

Domain	Text BLEU	Text METEOR	Triple precision	Triple recall	Triple F1	Predicate adherence
GSMArena	0	0	0	0	0	0
OpenWeather	0	0	0	0	0	0
OWID	0	0	0	0	0	0
Wikidata	0	0	0	0	0	0
Average	0	0	0	0	0	0

Table 5.17: Fine-tuned model results for the low learning-rate configuration.

Domain	Text BLEU	Text METEOR	Triple precision	Triple recall	Triple F1	Predicate adherence
GSMArena	0	0	0	0	0	0
OpenWeather	0	0	0	0	0	0
OWID	0	0	0	0	0	0
Wikidata	0	0	0	0	0	0
Average	0	0	0	0	0	0

Table 5.18: Fine-tuned model results for the higher-capacity configuration.

Domain	Text BLEU	Text METEOR	Triple precision	Triple recall	Triple F1	Predicate adherence
GSMArena	0	0	0	0	0	0
OpenWeather	0	0	0	0	0	0
OWID	0	0	0	0	0	0
Wikidata	0	0	0	0	0	0
Average	0	0	0	0	0	0

Table 5.19: Fine-tuned model results for the regularized-capacity configuration.

Domain	Text BLEU	Text METEOR	Triple precision	Triple recall	Triple F1	Predicate adherence
GSMArena	0	0	0	0	0	0
OpenWeather	0	0	0	0	0	0
OWID	0	0	0	0	0	0
Wikidata	0	0	0	0	0	0
Average	0	0	0	0	0	0

5.2 Surface realization

5.2.1 WebNLG dataset and evaluation protocol

Surface realization is evaluated on WebNLG, a benchmark in which RDF-style semantic triples are paired with one or more natural-language lexicalizations. The benchmark reader used in this work loads the English WebNLG release and preserves both the triples and their reference texts. During evolution, the training portion of WebNLG is used to provide fitness feedback. The final programs are evaluated independently on the held-out test portion, preventing the test references from being used during program search [9].

The evaluation is performed separately by WebNLG category. The program receives only the triples for each test entry. It must return one natural-language string. The baseline systems are a fine-tuned BART model, the *nlg-from-scratch* system of Lango et al., and a zero-shot prompted language model. The zero-shot prompt can be found in Appendix A.2.

The final evaluation reports reference-based, judge-based, and descriptive metrics. BLEU, BLEURT, and METEOR compare a generated text with the WebNLG references. SENLEN measures the absolute difference between the number of sentences in the generated text and the average number in the references. The three judge columns contain scores from Gemma 4 31B IT, Qwen 3.6, and Themis. The grammaticality, additions, and omissions judgments are performed with Gemma 4 31B IT. The prompts used for these judgments are provided in Appendix A.2. The final four columns are descriptive statistics: average sentence count, word count, character count, and number of input triples.

Higher values are preferable for BLEU, BLEURT, METEOR, judge scores, and grammaticality. Lower values are preferable for additions, and omissions; zero means that no addition or omission was detected. The closer to the value of one for SENLEN the better.

5.2.2 Evolution variants

All evolutionary runs use 1,000 iterations and Gemma 4 31B IT as the main program-generation model. The initial variant uses BLEU as the evaluator and provides one inspiration program in each prompt. It is intended to run on all WebNLG training categories.

The initial run was used to identify categories with different levels of difficulty. WrittenWork was selected as the best-working category, Building as an intermediate category, and Food as the most difficult category. Restricting subsequent variants to these three categories reduced the computational cost of the experiments while retaining distinct task difficulties.

The following variants were then evaluated:

BLEU without inspirations BLEU-based evaluation with no inspiration programs.

Gemma judge Gemma 4 31B IT used as an LLM judge instead of BLEU, with one inspiration.

DeepSeek judge LLM-as-a-judge evaluation with one inspiration and an occasional DeepSeek-V3.1 call with probability 0.2.

DeepSeek judge with four threads The preceding variant with four parallel evaluator threads.

Themis judge Themis as the evaluator, one inspiration, DeepSeek-V3.1 calls with probability 0.2, and four evaluator threads.

Llama judge Themis as the evaluator with four threads, while Llama 3.3 70B replaces Gemma 4 as the main evolutionary model.

Each run produces an executable surface-realization program. The program is then evaluated on the held-out WebNLG test set rather than being judged only by its online evolutionary fitness.

5.2.3 Evolution results

The following tables use the metric order shown in the evaluation protocol. Average rows are calculated over the available rows for the corresponding experiment.

Table 5.20: Initial BLEU-based evolution variant with one inspiration.

domain	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
Airport	0.4125	0.2992	0.7108	0.4649	0.9053	0.7179	0.8421	0.3789	0.0737	0.0000	0	0	0	0
Artist	0.3006	0.1340	0.6203	0.2661	0.8239	0.6936	0.7725	0.5872	0.0092	0.2018	0	0	0	0
Astronaut	0.5300	0.2160	0.7394	0.4858	0.9463	0.7561	0.8780	0.6463	0.0366	0.0366	0	0	0	0
Athlete	0.5632	0.4583	0.7828	0.3000	0.9360	0.8280	0.9240	0.8200	0.1000	0.0000	0	0	0	0
Building	0.4220	0.0381	0.6245	0.8043	0.9913	0.8652	0.9348	0.5870	0.0217	0.0000	0	0	0	0
CelestialBody	0.4962	0.3329	0.7172	0.4898	1.0000	0.8286	0.9510	0.6122	0.0000	0.0000	0	0	0	0
City	0.3463	0.1817	0.6536	0.3976	0.7831	0.6843	0.9157	0.8072	0.4578	0.0000	0	0	0	0
ComicsCharacter	0.2947	-0.0221	0.5241	0.5500	0.8000	0.6067	0.7733	0.3000	0.0000	0.5667	0	0	0	0
Company	0.4901	0.2592	0.7370	0.3788	0.9576	0.7818	0.9152	0.5758	0.0909	0.0000	0	0	0	0
Food	0.0764	0.0417	0.5173	0.8587	0.8652	0.4913	0.6348	0.0000	0.0435	0.0000	0	0	0	0
MeanOfTransportation	0.3523	0.2908	0.7313	0.2126	0.9241	0.7517	0.8966	0.5517	0.0345	0.0345	0	0	0	0
Monument	0.4950	0.2289	0.6997	0.8877	0.6043	0.4261	0.6652	0.3913	0.5870	0.0000	0	0	0	0
Politician	0.5059	-0.0405	0.7218	0.7701	1.0000	0.8138	0.9103	0.1724	0.0000	0.0000	0	0	0	0
SportsTeam	0.4638	0.1271	0.7756	0.7992	0.7818	0.4773	0.8136	0.4545	0.1136	0.0000	0	0	0	0
University	0.5473	0.1314	0.7261	0.2926	0.8733	0.7889	0.8644	0.4444	0.0889	0.0111	0	0	0	0
WrittenWork	0.5920	0.2027	0.7628	0.3488	0.9907	0.8326	0.9535	0.5116	0.0000	0.0000	0	0	0	0
Average	0.4305	0.1800	0.6903	0.5192	0.8864	0.7090	0.8528	0.4900	0.1036	0.0532	0	0	0	0

Table 5.21: BLEU-based evolution without inspirations.

domain	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
WrittenWork	0.5619	0.2174	0.7569	0.3488	0.9814	0.8698	0.9256	0.5349	0.0000	0.0000	2.3488	20.5116	110.0233	2.3488
Building	0.4273	0.0118	0.6418	0.8043	0.9870	0.7913	0.9174	0.2174	0.0000	0.0000	3.9565	34.6957	196.9348	4.5652
Food	0.1326	-0.1648	0.5218	0.8587	0.8565	0.5696	0.7783	0.2609	0.0000	0.0000	1.0000	24.9783	149.6957	4.5652
Average	0.3739	0.0215	0.6402	0.6706	0.9416	0.7435	0.8737	0.3377	0.0000	0.0000	2.4351	26.7285	152.2179	3.8264

Table 5.22: Gemma-as-a-judge evolution variant with one inspiration.

domain	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
WrittenWork	0.5405	0.2194	0.7667	0.3488	0	0	0	0	0	0	0	0	0	0
Building	0.4168	0.0214	0.6231	0.8043	0	0	0	0	0	0	0	0	0	0
Food	0.0671	-0.2275	0.4968	0.8587	0	0	0	0	0	0	0	0	0	0
Average	0.3414	0.0044	0.6289	0.6706	0	0	0	0	0	0	0	0	0	0

Table 5.23: LLM-judge evolution with occasional DeepSeek-V3.1 calls.

domain	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
WrittenWork	0.5499	0.2371	0.7615	0.3488	1.0000	0.9814	0.9953	0.8605	0.0000	0.0000	2.3488	20.3721	110.7907	2.3488
Building	0.3671	0.0777	0.6058	0.8043	1.0000	0.7565	0.9435	0.8478	0.0000	0.0000	3.3478	38.4565	211.5652	4.5652
Food	0.0970	-0.1622	0.4951	0.8587	1.0000	0.6826	0.8609	0.4348	0.0000	0.0000	1.0000	24.1522	146.7174	4.5652
Average	0.3380	0.0509	0.6208	0.6706	1.0000	0.8068	0.9332	0.7144	0.0000	0.0000	2.2322	27.6603	156.3578	3.8264

Table 5.24: LLM-judge evolution with occasional DeepSeek-V3.1 calls and four threads.

domain	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
WrittenWork	0.5348	0.2359	0.7386	0.4186	1.0000	0.9395	0.9953	0.8837	0.0000	0.0000	2.4186	19.8605	104.6744	2.3488
Building	0.4156	0.0063	0.6239	0.8043	1.0000	0.9000	0.9652	0.5870	0.0000	0.0000	3.3478	35.6522	197.8043	4.5652
Food	0.0828	-0.1881	0.5074	0.8587	1.0000	0.6826	0.9174	0.4565	0.0000	0.0000	1.0000	26.9348	159.4348	4.5652
Average	0.3444	0.0180	0.6233	0.6939	1.0000	0.8407	0.9593	0.6424	0.0000	0.0000	2.2555	27.4825	153.9712	3.8264

Table 5.25: Themis evolution with DeepSeek-V3.1 calls and four threads.

domain	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
WrittenWork	0.5487	0.1735	0.7553	0.4186	0.9116	0.8791	0.9349	0.8372	0.1163	0.1163	2.2791	19.2791	106.1395	2.3488
Building	0.4106	-0.0067	0.5928	0.8043	0.8957	0.8217	0.9130	0.5870	0.1739	0.2609	3.3478	32.9348	179.4348	4.5652
Food	0.0880	-0.0514	0.4279	0.8587	0.6696	0.4870	0.7435	0.6739	0.2174	0.5000	1.0000	24.2174	142.2609	4.5652
Average	0.3491	0.0385	0.5920	0.6939	0.8256	0.7293	0.8638	0.6994	0.1692	0.2924	2.2090	25.4771	142.6117	3.8264

TABLE 5.26: Themis evolution with Llama 3.3 70B replacing Gemma 4 as the evolutionary model.

domain	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
WrittenWork	0.5077	-0.0745	0.7263	0.3488	0.5953	0.4279	0.7628	0.1628	0.0698	0.0930	2.3488	19.6512	104.1395	2.3488
Building	0.2395	-0.4696	0.3725	0.7971	0.4696	0.3522	0.4739	0.0217	0.1304	1.0000	3.3478	21.0217	115.5652	4.5652
Food	0.0419	-0.1023	0.4081	0.8587	0.3826	0.2696	0.6522	0.0000	0.1087	0.5870	1.0000	20.1739	125.4783	4.5652
Average	0.2630	-0.2155	0.5023	0.6682	0.4825	0.3499	0.6296	0.0615	0.1030	0.5600	2.2322	20.2823	115.0610	3.8264

5.2.4 Baseline systems

The baseline scores are collected from the fine-tuned BART, zero-shot and the Lango et al. method. The average is calculated over the available domain results for each baseline.

Table 5.27: Baseline scores across WebNLG domains.

method	domain	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
nlg-from-scratch	all domains	0.1702	-0.3874	0.5205	1.0175	0.4108	0.4811	0.7552	0.1783	0.0889	0.0742	2.5414	21.7471	146.6654	3.1946
BART	all domains	0.5077	0.2599	0.7163	0.5157	0.6366	0.5933	0.7899	0.8366	0.3142	0.1863	2.1679	138.9440	23.7167	3.5205
zero-shot	all domains	0.5019	0.3443	0.7246	0.5274	0.9954	0.9743	0.9847	0.9861	0.0045	0.0022	1.7307	132.0601	21.5757	3.5205

5.2.5 Three-domain comparison

The final comparison with the baselines. The baseline rows below contain the average over WrittenWork, Building, and Food; and the OpenEvolve's variant is (placeholder).

Table 5.28: Average scores on WrittenWork, Building, and Food.

method	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
Fine-tuned BART	0.2063	-0.4489	0.5498	1.9138	0.2316	0.3529	0.6380	0.0694	0.4291	0.1836	4.3887	30.8329	199.2032	3.3999
nlg-from-scratch	0.5628	0.2398	0.7126	0.7414	0.6707	0.5964	0.8123	0.7878	0.2782	0.1122	2.3025	147.8967	27.3637	3.8264
Zero-shot	0.4781	0.2310	0.6720	0.6706	0.9848	0.9660	0.9826	0.9778	0.0000	0.0000	2.2322	141.2164	25.2174	3.8264
Best OpenEvolve variant	0	0	0	0	0	0	0	0	0	0	0	0	0	0

5.3 Whole system evaluation

The whole-system evaluation combines the meaning-representation and surface- realization stages into one experimental pipeline. Unlike the preceding experiments, which evaluate the two stages separately, this evaluation measures whether errors introduced while converting raw structured data into triples affect the final generated text.

5.3.1 Data construction

Two new collections are downloaded with quintd for each of the four domains: GSMArena, OpenWeather, OWID, and Wikidata. The first collection is denoted by S_3 and contains 1,000 raw structured instances per domain for training the evolutionary programs. The second collection is denoted by S_4 and contains another 1,000 raw structured instances per domain for final testing. The two collections are downloaded independently so that no raw instance from the test collection is available during evolution.

The raw instances in both collections are processed with the selected best fine-tuned Gemma 4 model from the preceding fine-tuning experiments. For each instance, the model produces a natural-language reference and the associated normalized semantic triples in a single output. Thus, both S_3 and S_4 contain aligned pairs of the form

$$(T_i, r_i),$$

where T_i is the generated set of triples and r_i is the corresponding reference text. The model is selected before this evaluation and remains frozen while the whole-system experiments are performed. The generated references are therefore used as domain-specific evaluation targets, rather than as additional inputs to the surface-realization programs.

5.3.2 Evolution and evaluation protocol

For each domain, the selected best-working OpenEvolve variant is run using the S_3 collection. The evolved program receives only T_i and must generate a natural-language text. Its fitness is computed by comparing the generated text with r_i on the 1,000 training instances. A separate best program is retained for each domain. The S_4 triples and reference texts are not used during program generation, fitness calculation, or program selection.

After evolution, each domain-specific program is executed on all 1,000 instances in the corresponding S_4 test collection. The generated texts are evaluated using the same protocol as the WebNLG surface-realization experiments. BLEU, BLEURT, and METEOR compare the generated text with its reference. SENLEN measures the difference between the generated and reference sentence counts. Gemma 4, Qwen, and Themis provide judge scores, while grammaticality, additions, and omissions are evaluated using the corresponding judge prompts. Average sentence count, word count, character count, and number of input triples are reported as descriptive statistics.

In this experiment, the reference triples are produced by the selected fine-tuned model rather than annotated independently by humans. Consequently, the evaluation measures the quality of the complete data-to-text pipeline with respect to the distilled references. It does not separately estimate the semantic precision or recall of the fine-tuned model; those properties are reported in the fine-tuning experiments.

TABLE 5.29: Whole-system evaluation on the independent S_4 test collections.

domain	BLEU	BLEURT	METEOR	SENLEN	Gemma	Qwen	Themis	gram.	add.	om.	sent.	words	chars	triples
GSMArena	0	0	0	0	0	0	0	0	0	0	0	0	0	0
OpenWeather	0	0	0	0	0	0	0	0	0	0	0	0	0	0
OWID	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Wikidata	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Average	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Chapter 6

Discussion

Chapter 7

Summary

Bibliography

[1] Ron Artstein and Massimo Poesio. Survey article: Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4):555–596, 2008.

[2] Satanjeev Banerjee and Alon Lavie. Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. `https://aclanthology.org/W05-0909/`, 2005.

[3] Thiago Castro Ferreira, Chris van der Lee, Emiel van Miltenburg, and Emiel Krahmer. Neural data-to-text generation: A comparison between pipeline and end-to-end architectures. `https://arxiv.org/abs/1908.09022`, 2019.

[4] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[5] Richard Cyganiak, David Wood, and Markus Lanthaler. Rdf 1.1 concepts and abstract syntax. `https://www.w3.org/TR/rdf11-concepts/`, 2014.

[6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. `https://arxiv.org/abs/2305.14314`, 2023.

[7] Bhuwan Dhingra, Manaal Faruqui, Ankur Parikh, Ming-Wei Chang, Dipanjan Das, and William Cohen. Handling divergent reference texts when evaluating table-to-text generation. `https://arxiv.org/abs/1906.01081`, 2019.

[8] Ondrej Dušek and Zdenek Kasner. Evaluating semantic accuracy of data-to-text generation with natural language inference. `https://arxiv.org/abs/2011.10819`, 2020.

[9] Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. The WebNLG challenge: Generating text from RDF data. In *Proceedings of the 10th International Conference on Natural Language Generation*, pages 124–133. Association for Computational Linguistics, 2017.

[10] Albert Gatt and Emiel Krahmer. Survey of the state of the art in natural language generation: Core tasks, applications and evaluation. `https://arxiv.org/abs/1703.09902`, 2018.

[11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. `https://arxiv.org/abs/2106.09685`, 2021.

[12] Xinyu Hu, Li Lin, Mingqi Gao, Xunjian Yin, and Xiaojun Wan. Themis: A reference-free nlg evaluation language model with flexibility and interpretability. `https://arxiv.org/abs/2406.18365`, 2024.

[13] Zdeněk Kasner and Ondřej Dušek. Beyond traditional benchmarks: Analyzing behaviors of open llms on data-to-text generation. `https://arxiv.org/abs/2401.10186`, 2024.

[14] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. `https://arxiv.org/abs/1412.6980`, 2014.

[15] Klaus Krippendorff. Reliability in content analysis: Some common misconceptions and recommendations. *Human Communication Research*, 30(3):411–433, 2004.

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. https://arxiv.org/abs/2309.06180, 2023.

[17] Mateusz Lango and Ondrej Dusek. LLM agents implement an NLG system from scratch: Building interpretable rule-based RDF-to-text generators. https://aclanthology.org/2025.emnlp-industry.142/, 2025.

[18] Sourab Mangrulkar, Sylvain Gugger, Lysak Lysak, et al. PEFT: Parameter-efficient fine-tuning of billion-scale models on low-resource hardware. https://github.com/huggingface/peft, 2022.

[19] Amit Moryossef, Yoav Goldberg, and Ido Dagan. Step-by-step: Separating planning from realization in neural data-to-text generation. https://arxiv.org/abs/1904.03396, 2019.

[20] Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/AlphaEvolve.pdf, 2025.

[21] Jekaterina Novikova, Ondrej Dušek, Amanda Cercas Curry, and Verena Rieser. Why we need new evaluation metrics for NLG. https://arxiv.org/abs/1707.06875, 2017.

[22] Chinonso Cynthia Osuji, Thiago Castro Ferreira, and Brian Davis. A systematic review of data-to-text nlg. https://arxiv.org/abs/2402.08496, 2024.

[23] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Ray, et al. Training language models to follow instructions with human feedback. https://arxiv.org/abs/2203.02155, 2022.

[24] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: A method for automatic evaluation of machine translation. https://aclanthology.org/P02-1040/, 2002.

[25] Ehud Reiter and Robert Dale. Building applied natural language generation systems. https://www.researchgate.net/publication/2839784_Building_applied_natural_language_generation_systems, 1997.

[26] Thibault Sellam, Dipanjan Das, and Ankur Parikh. BLEURT: Learning robust metrics for text generation. https://arxiv.org/abs/2004.04696, 2020.

[27] Asankhaya Sharma. Openevolve: An open-source evolutionary coding agent. https://github.com/algorithmicsuperintelligence/openevolve, 2025.

[28] Gemma Team et al. Gemma: Open models based on gemini research and technology. https://arxiv.org/abs/2403.08295, 2024.

[29] Gemma Team et al. Gemma 4 technical report. https://arxiv.org/abs/2607.02770, 2026.

[30] Chris van der Lee, Emiel Krahmer, and Sander Wubben. Automated learning of templates for data-to-text generation: Comparing rule-based, statistical and neural methods. https://aclanthology.org/W18-6504/, 2018.

[31] Huanting Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. Ai agentic programming: A survey of techniques, challenges, and opportunities. https://arxiv.org/abs/2508.11126, 2025.

[32] Zhuoer Wang, Marcus Collins, Nikhita Vedula, Simone Filice, Shervin Malmasi, and Oleg Rokhlenko. Faithful low-resource data-to-text generation through cycle training. https://aclanthology.org/2023.acl-long.160/, 2023.

[33] Jędrzej Warczyński, Mateusz Lango, and Ondrej Dusek. Leveraging large language models for building interpretable rule-based data-to-text systems. https://arxiv.org/abs/2502.20609, 2024.

[34] Sam Wiseman, Stuart M. Shieber, and Alexander M. Rush. Challenges in data-to-document generation. https://arxiv.org/abs/1707.08052, 2017.

[35] Jiannan Xiang, Zhengzhong Liu, Yucheng Zhou, Eric Xing, and Zhiting Hu. ASDOT: Any-shot data-to-text generation with pretrained language models. https://aclanthology.org/2022.findings-emnlp.136/, 2022.

[36] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert. https://arxiv.org/abs/1904.09675, 2020.

Appendix A

Prompts

A.1 Evolution prompts for surface realization

The following prompts are the experiment-specific templates used to evolve the surface-realization program. Values in braces are filled at runtime by OpenEvolve. The templates are shown separately because the final user message is assembled from the mutation template, the current program, evaluation metrics, evolution history, and optional artifacts.

A.1.1 Evolution system prompt

```
You are an expert data engineer specializing in converting data to text. You will be given the
Triples:
(Antwerp | cityServed | Antwerp International Airport)
(Belgium | country | Antwerp)
(City of Brussels | capital | Belgium)
Example sentence:
"Antwerp International Airport serves the city of Antwerp which is in Belgium, where the capita

The 'predict' function returns that sentence as a string. Below is the list of all possible pre

{triples}
```

A.1.2 Evolution user prompt

The custom user template for diff-based evolution is:

```
# Current Program Information
{metrics}

{artifacts}

{evolution_history}

# Current Program
```{language}
{current_program}
```
```

```
‘‘‘

# Task
Suggest an improvement to the current program that will improve the combined_score.
Different solutions with similar combined_score score but different ideas are valuable.
Creating one complex sentence from all the given triples will award greater score as well.

You MUST use the exact SEARCH/REPLACE diff format shown below to indicate changes:

<<<<<<< SEARCH
# Original code to find and replace (must match exactly, indentations and endlines as well)
=======
# New replacement code
>>>>>>> REPLACE

Example of a valid diff format:
<<<<<<< SEARCH
    best_x = 0
    best_y = 0
=======
    # Initialize with a random point
    best_x = np.random.uniform(bounds[0], bounds[1])
    best_y = np.random.uniform(bounds[0], bounds[1])
>>>>>>> REPLACE

The SEARCH section must exactly match the code in the current program, even with it’s indentati
Do not provide explanations, only the SEARCH/REPLACE section.
IMPORTANT: Do not rewrite the entire program - focus on targeted improvements.
```

The placeholders are expanded as follows: `metrics` contains the parent's recorded metrics, `artifacts` contains the optional artifact message shown in Appendix A.1.4, `evolution_history` contains previous attempts and selected programs, and `current_program` contains the parent source code. In the reported run, the top-program and inspiration subsections were disabled, but the inspiration slot was enabled separately.

A.1.3 Evolution-history components

The user prompt can contain the following runtime-generated history components. They are included here to make the composition of the complete user message explicit.

```
{inspirations_section}

### Attempt {attempt_number}
- Changes: {changes}
- Metrics: {performance}
- Outcome: {outcome}
```

```
### Program {program_number} (Score: {score})
```{language}
{program_snippet}
```
Key features: {key_features}

Inspiration Programs

This is a different program that might have a different approach on how to solve the problem. U

{inspiration_programs}

Inspiration {program_number} (Score: {score})
```{language}
{program_snippet}
```
```

A.1.4 Artifact message

Artifacts returned by the evaluator are wrapped before being inserted into the user prompt. The wrapper has the following form:

```
## Evaluator Artifacts

### {artifact_key}
```
{artifact_content}
```
```

The artifact content is truncated to the configured maximum size and passed through the artifact security filter. For example, a low-scoring LLM-judge artifact has the following structure:

```
The program got scored by the judge with the score {rating}. The input triples were:
{subject} | {predicate} | {object}

The generated text was:
{generated_text}

The judge review is:
{review}

Based on the review try to understand why the program might have performed poorly and improve t
```

A.1.5 Repair prompts

When no valid diff is extracted, the following repair messages are used. The second user message is also used when a syntactically valid diff does not apply any change to the parent source.

```
SYSTEM:
```

```
You are an expert in fixing your collegues code. You know that the code should be in format:
<<<<<<< SEARCH
# Original code to find and replace (must match exactly, indentations and endlines as well)
=======
# New replacement code
>>>>>>> REPLACE
Provided the incorrect format of SEARCH/REPLACE fix it to a correct format.

USER:
The original code was:
{parent_code}
An incorrect diff format was detected in this change:
{llm_response}
Please fix it to the correct format.

SYSTEM:
You are an expert in fixing your collegues code. You know that in order for the SEARCH/REPLACE

USER:
The provided diff did not apply any changes to the original code:
{llm_response}
Given the original code please modify the SEARCH section to ensure it matches exactly the searc
{parent_code}
```

The spelling in these blocks is preserved from the implementation because the appendix documents the actual prompts sent to the model.

A.2 LLM-as-a-judge prompt

A.2.1 Meaning-representation pipeline prompts

The meaning-representation pipeline evaluation uses four independent criteria: summary, completeness, faithfulness, and omissions. Each criterion is scored with a separate prompt on an integer scale from 1 (worst) to 5 (best). The judge is instructed to assess only the relationship between the supplied source and output, without using external knowledge. The same criteria are used for both transitions, but the source and output supplied to the judge depend on the transition:

Data to text The source is the original structured data instance and the output is the generated natural-language reference text.

Text to triples The source is the generated reference text and the output is the generated semantic triples.

The task-specific part of the system prompt is instantiated as follows:

```
You are a strict judge evaluating a {conversion} conversion.
You will see: (1) {source_label}, and (2) {output_label}.
```

```
{criterion_guideline}

Use only the provided source and output. Assign one integer score from 1 to 5.
Return ONLY JSON with this exact schema:
{"score": <integer 1-5>, "reason": "<one short sentence>"}
```

For the data-to-text transition, `conversion` is "data-to-text", `source_label` is "the original structured data instance", and `output_label` is "the natural-language text generated from it". For the text-to-triples transition, the corresponding values are "text-to-triples", "the natural-language reference text", and "the semantic triples generated from it". The four criterion guidelines are:

Summary

```
Criterion: Summary
Evaluate whether the output is a concise, coherent summary of the source. It
should avoid repeating the same concept with different words, be significantly
shorter than a detailed or repetitive source when possible, preserve the core
message, and function as a summary rather than a rewritten essay or a
disjointed list of facts.

5: The output is a clear, coherent, and significantly more concise summary. It
preserves the source's core message without redundant restatement and does not
read like an essay or a disconnected list.
4: The output is a good summary and preserves the core message, but has minor
redundancy, slight unnecessary length, or a small organizational weakness.
3: The output is recognizably a summary, but is noticeably repetitive, too
verbose, list-like, or weakly organized; its main message remains usable.
2: The output is a poor summary: it substantially repeats or rewrites the
source, is very verbose or disjointed, and represents the core message only
partly.
1: The output does not function as a summary. It is mostly irrelevant,
repetitive, disjointed, or fails to communicate the source's core message.
```

Completeness

```
Criterion: Completeness
Evaluate whether the output includes all main points and all critical facts,
constraints, context, entities, relationships, values, and conclusions from
the source. Judge coverage, not whether the output contains unsupported
information.

5: Every main point and every critical fact, constraint, context element, and
conclusion needed to understand the source is represented in the output. No
important gap remains.
4: All main points and critical context are present, but one or a few
secondary, non-critical details are missing.
3: Most main points are present, but at least one important fact or context
element, or several secondary details, are missing.
```

```
2: Multiple main points or a critical constraint, context element, or
conclusion is missing, so the output gives a substantially incomplete account.
1: Little relevant source information is represented, or the source's main
message is largely absent.
```

Faithfulness

```
Criterion: Faithfulness
Evaluate whether every claim in the output is directly supported by the source
and whether entities, relationships, values, qualifiers, and their meaning
are represented accurately. Penalize hallucinations, fabrications,
contradictions, and misinterpretations. Do not penalize information that is
merely omitted; evaluate omissions under Completeness and Omissions.

5: Every claim is grounded in the source, and all represented facts preserve
the source's meaning. There are no hallucinations, contradictions, or material
distortions.
4: The output is fully usable and essentially faithful, with at most a minor
imprecision that does not change the meaning and no material unsupported claim.
3: Most claims are grounded, but there is one material error or distortion, or
several minor unsupported or inaccurate claims.
2: There are multiple material errors, hallucinations, contradictions, or a
substantial misinterpretation of the source.
1: The output is mostly ungrounded or contradicts the source, so it cannot be
considered a reliable representation.
```

Omissions

```
Criterion: Omissions
Evaluate the consequences of what the output leaves out. Omitting trivial
examples, background fluff, and repetitive anecdotes is appropriate and must
not lower the score. Lower the score only when omissions remove critical
context, change the meaning, hide a major conclusion or constraint, or create
a misleading imbalance or bias.

5: Only trivial, redundant, or non-essential background material is omitted.
No omission changes the meaning, hides important context, or introduces a
meaningful bias.
4: One or a few minor omissions exist, but the meaning and conclusions remain
intact and the selection of included information is not meaningfully biased.
3: An important but non-central detail, or enough supporting context to create
a mild imbalance, is omitted; the overall message remains understandable.
2: A critical context element, major conclusion, important constraint, or
representative point is omitted, changing the meaning or creating clear bias.
1: Extensive omissions distort the core message or make the output strongly
misleading or one-sided; most important source information is absent.
```

A.2.2 Surface-realization judge prompts

For every non-empty generated realization, the evaluator sends the input triples and the generated text to the configured LLM judge. The reference texts are not included in this request. The prompt used by the evaluator is shown below; `source` is the formatted list of triples and `target` is the generated text.

```
###Instruction###
Please act as an impartial and helpful evaluator for natural language generation (NLG), and the
Your task is to evaluate the quality of text similarity strictly based on the given evaluation
Begin the evaluation by providing your analysis concisely and accurately, and then on the next
You MUST keep to the strict boundaries of the evaluation criterion and focus solely on the issu
Make sure you read and understand these instructions, as well as the following evaluation crite

###Evaluation Criterion###
Accuaracy and number of sentences: The generated text must accurately reference the triples. Th

###Data###
The triples in the form (subject, predicate, object):
{source}

The generated text:
{target}

###Your Evaluation###
```

The response is parsed as a review followed by a rating from 1 to 5. The ratings are averaged and divided by 5 before they are used as the evolutionary fitness.

A.2.3 Grammaticality prompt

The following criterion is used for the binary grammaticality score. The judge returns 1 for a grammatically correct realization and 0 otherwise.

```
Grammatical correctness: You should assess the grammatical correctness of the
resulting text. Do not take any other factors into account. Do not make
assumptions or consider external knowledge not present in the provided context.
Identify only errors relating to the grammaticality of the text. Do not
consider aspects such as fluency, omissions or hallucinations.

Return 1 if the text is grammatically correct and 0 if it is not. The rating
must be the integer 0 or 1.
```

A.2.4 Omissions prompt

The omissions score measures whether any input triples were not verbalized. A score of 0 indicates that no omission was detected, while 1 indicates that at least one omission was detected.

```
Omissions: You should assess the omissions in the resulting text; in other
words, check whether any of the input triples were not verbalised. You can
```

```
perform the task by iterating over the input triples and checking if each is
present in the output. Do not take any other factors into account. Do not make
assumptions or consider external knowledge not present in the provided context.
Identify only omissions. Do not consider grammaticality, fluency or the
addition of new facts.

Return 0 if there are no omissions and 1 if there are. The rating must be the
integer 0 or 1.
```

A.2.5 Additions prompt

The additions score measures whether the realization introduces facts that are not supported by the input triples. A score of 0 indicates that no addition was detected, while 1 indicates that at least one addition was detected.

```
Additions: You should assess the addition of new facts in the resulting text
which were not present in the input triples. Carefully read the text and check
whether the facts mentioned are present in the input triples. Do not take any
other factors into account. Do not make assumptions or consider external
knowledge not present in the provided context. Identify only additions. Do not
consider grammaticality, fluency or omissions.

Return 0 if there are no additions and 1 if there are. The rating must be the
integer 0 or 1.
```

A.2.6 Zero-shot prompt

The zero-shot prompt is used to convert the input triples into a natural language realization without any additional context. The prompt is shown below;

```
You are an expert data-to-text generator. Verbalize the given RDF triples
into a single fluent, grammatically correct English sentence that conveys
every fact and adds no extra information.

Convert these RDF triples (subject | predicate | object) into one
natural-language sentence.

Triples:
{triples}

Output only the sentence.
```

A.3 Text-First Extraction with Normalization

This section contains the complete text of all system prompts used in the Text-First Extraction with Normalization method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: {"text":"..."}. Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

Triples-From-Text System Prompt (P_{tr})

You extract semantic triples from natural language text. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...","predicate":"...","object":"..."}]}. Rules: preserve the full meaning of the input text, avoid duplicate triples, and keep predicates in lower_snake_case when possible.

Candidate-Pairs System Prompt

You find candidate predicate pairs that may be semantically equivalent. Return ONLY JSON with schema: {"pairs":[{"predicate_a":"...","predicate_b":"..."}]}. Rules: include only pairs with high likelihood of strict semantic equivalence; do not include pairs that are merely related; do not include duplicates or self-pairs.

Strict-Equivalence System Prompt

You decide if two predicates are semantically equivalent in meaning. Return ONLY JSON with schema: {"equivalent": true|false, "confidence": 0.0-1.0, "reason": "..."}. Be strict: equivalent only if they can be safely merged in a knowledge graph.

A.4 Blueprint-Iterative Refinement

This section contains the complete text of all system prompts used in the Blueprint-Iterative Refinement method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: {"text":"..."}. Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

Blueprint-Creation System Prompt ($P_{\text{blueprint}}$)

You design extraction rules for a domain. Return ONLY JSON with schema: {"predicates":["..."], "example_triples":[{"subject":"...","predicate":"...","object":"..."}], "example_text":"...", "guidelines":["..."]}. Rules: include precise and reusable predicates, provide concrete example triples with strict formatting, provide an example text matching those triples, and give practical transformation guidelines.

Blueprint-Feasibility System Prompt (P_{feas})

You evaluate whether given extraction rules are sufficient to extract triples from a text. Return ONLY JSON with schema: {"possible":true|false,"reason":"...","rule_gaps":["..."]}.

Blueprint-Revision System Prompt (P_{rev})

You revise extraction rules so triples can be produced from a text. Return ONLY JSON with schema: {"predicates":["..."], "example_triples":[{"subject":"...","predicate":"...","object":"..."}], "example_text":"...", "guidelines":["..."]}. Keep existing high-quality rules, but add or adjust what is necessary to cover the failing case.

Triples-From-Text-With-Blueprint System Prompt ($P_{\text{tr\&}R}$)

You extract semantic triples from natural language text using provided rules. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...","predicate":"...","object":"..."}]}. Follow rule guidelines and mimic the formatting style from example triples. Prefer predicates from the rule predicate list whenever possible.

A.5 Evolving Catalog

This section contains the complete text of all system prompts used in the Evolving Catalog method (Section 3.2).

Text-Generation System Prompt (P_{txt})

You convert one structured data instance into concise natural language. Return ONLY JSON with schema: {"text":"..."}. Capture the most important information, trends, extremes, and notable conditions. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

First-Instance System Prompt (P_{first})

You extract semantic triples from text. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...","predicate":"...","object":"..."}]}. Use concise predicate labels in lower_snake_case when possible and avoid duplicates.

Catalog-Guided System Prompt (P_{cat})

You extract semantic triples from text using a predicate catalog with examples. Return ONLY JSON with this exact schema: {"triples":[{"subject":"...","predicate":"...","object":"..."}]}. Use only predicates from the provided catalog when possible. Introduce a new predicate only when none of the catalog predicates can describe the fact.

Appendix B

Prompts for Fine-tuned Model

System Prompt for Joint Text and Triple Generation

You convert one structured data instance into concise natural language and a corresponding set of RDF semantic triples. Return ONLY JSON with schema: {"text":"...","triples":[{"subject":"..
Capture the most important information, trends, extremes, and notable conditions. Use concise predicate labels in lower_snake_case when possible and avoid duplicates. If the instance contains a time series (for example a weather forecast), summarize it at a high level instead of listing every point.

User Message Template

Create a concise natural-language summary of this ONE data instance and extract its semantic triples.

instance_context=`{serialized JSON instance}`

Return JSON only.

Assistant Message (Expected Output)

{"text": "<generated natural language text>", "triples": [{"subject": "...", "predicate": "...", "object": "..."}, ...]}

Appendix C

Initial Program

The following program is the complete initial source used to start surface realization evolution. The code outside the evolution block defines the data type and the public interface; the marked block is the part that OpenEvolve is allowed to modify.

```
from dataclasses import dataclass

@dataclass
class Triple:
    subject: str
    predicate: str
    object: str

# EVOLVE-BLOCK-START

def predict(triples: list[Triple]) -> str:
    return f"The {triples[0].predicate} of {triples[0].subject} is {triples[0].object}."

# EVOLVE-BLOCK-END
```

Poznan University of Technology
Faculty of Computing
Institute of Computing Science

Typeset using LaTeX in Computer Modern.

BibTeX:

```
@mastersthesis{ key,
    author = "Dominik Maćkowiak",
    title = "{Interpretable methods for data-to-text generation}",
    school = "Poznan University of Technology",
    address = "Pozna{\'n}, Poland",
    year = "2026",
}
```